

موضوع

- اهمیت ساختار داده و تحلیل زمانی کد داده

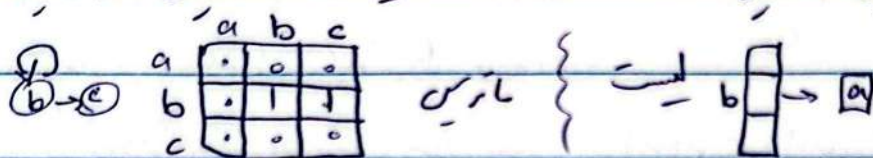
- تحلیل زمانی مرتب سازی و جابجایی

- مجموعه

آشنایی با آرایه (array)، لیست (queue)، پشته (stack) و کاربرد آنها

آشنایی با Linked List

- آشنایی با ماتریس هسلیگ و لیست هسلیگ (نمایش گراف)



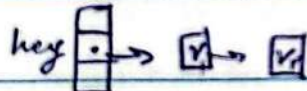
- الگوریتم بیاض گراف DFS، BFS

- درخت (گراف همبند بدون حلقه) و درخت جستجو (BST)

- درخت متوازن Red Black

Augmented Data structures

Max/Min Heap (درخت درخت)



Hashable و Hash

- تکنیک های الگوریتم Divide & Conquer (تقسیم، حل بازگشتی، ترکیب)

Floyd & Dynamic program

Greedy & min, knapsack, Dijkstra

ارزایی: میان برد / لا نه / میان / تری / لا نه / میان / تری

آزمایش

الگویم: کلمات مرتب برای انجام واحد منطقی از کار.

ساختار دانه: حینش دانه ها در حافظه

ADT: Abstract data type ہے کیسے کریں۔ ساختار درجہ (کلاس)

کلیں زبانیں سودو کو دے C_1 کلاس (س) $\rightarrow S = 0$

for $i = 1$ to n , do $\frac{(n+1)(n)}{2}$ (نہ مانے)

اعمال ریاضی و assignments مناسب این، $\left\{ S \rightarrow S + I \rightarrow \left(\sum_{i=1}^n C_i \right) \right\}$

لے کتوات پایہ: کتوات پایہ کہ در شش ہاں

مختلف در تعداد نایب کلاس می‌باشد

انجام می کشید

For

$$\Rightarrow T(n) = C_1 + C_2(n+1) + C_2(n)$$

در منابع هم است. مثل من خواهم

۱- تابع در ∞ حدیابی کنیم.

$$T(x) = ax + b \quad \text{Line}$$

جس

4. If is_n then $jump(l_r)$

زماں لہرا تاہی از اندازہ حسن است.

$$S_5 S_4 i$$

انٹازہ وروسی (ساتر مسئلہ) بلایر

$$i = i + 1$$

۱) تعداد بیت های لازم برای

jump 2.

(ذخیره سازی داده ورودی)

c_r : exit

Header حصہ یک بار ستر از body (تایرنگار)

سوال: دو عدد n و m را جمع می‌کنیم. $\log_2 \text{bit}$ سایر هر دو وابسته به n

مثال ۲: عدد هفت را جمع می‌کنیم. (است)

$\Rightarrow r \times r \times r \Rightarrow 4 \text{ n bit}$

آفینہر

$S = 0 \rightarrow C_1$

for $i = 1$ to n do $\rightarrow (n+1) C_2$ جمله header

for $j = 1$ to i do $\rightarrow C_3 \sum_{i=1}^n \sum_{j=1}^i 1$

$S = S + j \rightarrow C_4 \sum_{i=1}^n \sum_{j=1}^i i$ جمع i تا i

$$T(n) = C_1 + C_2(n+1) + C_3 \sum_{i=1}^n \sum_{j=1}^i 1 + C_4 \sum_{i=1}^n \sum_{j=1}^i i$$

$$= C_1 + C_2(n+1) + C_3 \sum_{i=1}^n (i+1) + C_4 \sum_{i=1}^n i$$

$$= C_1 + C_2(n+1) + C_3 \frac{n(n+3)}{2} + C_4 \frac{n(n+1)}{2}$$

$$\rightarrow \sum_{i=1}^n i+1 = 1+2+3+\dots+n+1 = n + \frac{n(n+1)}{2} = \frac{n(n+3)}{2}$$

Insertion Sort

تکلیف الگوریتم مرتب سازی

چندین داده هم نوع و فیزیکی نیست بهم (جستجو در آرایه A آرایه A آرایه A)

0	1	2	3	4
1	2	5	4	7

از سمت چپ آرایه را در نظر می گیریم و آن را مرتب می کنیم. یعنی هر بار اندک
هتان مرتب است و خانه $i+1$ را طوری قرار می دهیم که $i+1$ از خانه مرتب باشد.
خانه $i+1$ را key نامیده و این کار را برای key از $n-1$ تا n تکرار می کنیم.

sorted

key →

2	5	1	4	7
---	---	---	---	---

آزمایش

InsertionSort (A, n):

For $j = 2$ to n do: $\rightarrow C_n$ (از 2 تا n فرقی ندارد)

key = A[j] $\rightarrow C_1(n-1)$

$i = j - 1 \rightarrow C_1(n-1)$

while key < A[i] and $i > 0$ do:

A[i+1] = A[i] $\rightarrow C_2 \sum_{j=2}^n (t_j)$ $\rightarrow C_2 \sum_{j=2}^n (t_{j+1})$

$i = i - 1 \rightarrow C_1 \sum_{j=2}^n (t_j)$

A[i+1] = key $\rightarrow C_1(n-1)$

t_j نشان می دهد در کدام j ام جا shift نیاز است. (از جایی که key کوچکتر است.)

$$T(n) = C_1 n + \dots + C_2 \sum_{j=2}^n (t_{j+1}) + C_1 \sum_{j=2}^n t_j =$$

$$= a_n + b + C_1 \sum_{j=2}^n t_j + C_2 \sum_{j=2}^n (t_{j+1})$$

کمال $\rightarrow$ worst case, Best case: Arg

$$\rightarrow 0 \leq t_j \leq j-1$$

$$\text{Best case: } a_n + b + 0 + C_2 \sum_{j=2}^n 1 = a'n + b'$$

$$\text{Worst case: } a_n + b + (C_2 + C_1) \sum_{j=2}^n (j-1) + C_2 \sum_{j=2}^n j$$

$$= a_n + b + (C_2 + C_1) \frac{n(n-1)}{2} + C_2 \frac{(n-1)(n+2)}{2}$$

$$= a''n^2 + b'n + c \quad \text{quadratic} / 2 \text{ در}$$

$$\bar{X} = E(X) = \sum x_i \cdot pr(x = x_i)$$

X متغیر تصادفی $\rightarrow$ x_i احتمال X برابر x_i شود

آزمین

می‌توان اثبات کرد در حالت متوسط (اسیدیهایی) $T_z = \frac{n-1}{2}$ است و داریم:

$$E[T_z] = \frac{n-1}{2} \text{ و } E[T(n)] = a^*n^2 + b^*n + c^*$$

پس insertion sort به طور متوسط تابع درجه ۲ است. یعنی اگر تعداد ورودی ۳ برابر شود انتظاری در زمان ۹ برابر شود.

معمولاً اثبات درستی الگوریتم با استقر است. ثابت می‌کنیم برای $n=1$ درست است و اگر برای n درست باشد برای $n+1$ درست است.

به این روش نگاه به مسئله incremental یا online می‌گوئیم که ابتدا ورودی کم را بررسی می‌کنیم و فرقی نمی‌کنیم هر بار با افزودن ورودی جدید چگونه باید مسئله را حل کرد. به کاربردی در stream ها

مرتب‌سازی حسابی: هر بار max را در آخر قرار داده و نگه‌دار می‌کنیم.

Bubble Sort (A, n): (آزیم از n فرض شده)

for $i=1$ to $n-1$ do: $\rightarrow C_1 \cdot n$
تعداد max - تعدادهای بزرگتر

~~$max = A[i]$~~ $\rightarrow n - (i-1)$

for $j=2$ to $n-i+1$ do: $\rightarrow C_2 \sum_{i=1}^{n-1} (n-i+1)$

~~$swap(A[j], A[j-1])$~~

if $A[j] < A[j-1]$ then: $C_3 \sum_{i=1}^{n-1} (n-i)$

$swap(A[j], A[j-1])$ $C_4 \sum_{i=1}^{n-1} (n-i)$

Best/Worst Case باید بررسی شود

آزیم

$$\begin{aligned}
 T(n)_{\text{best case}} &= C_1 n + C_2 \sum_{i=1}^{n-1} (n-i+1) + C_3 \sum_{i=1}^{n-1} (n-i) \\
 &= C_1 n + C_2 \frac{(n+1)(n-1)}{2} + C_3 \frac{(n)(n-1)}{2} \\
 &= a'n^2 + b'n + c' \rightarrow \text{درجه ۲}
 \end{aligned}$$

$$T(n)_{\text{worst case}} = T(n)_{\text{best case}} + C_4 \frac{n(n-1)}{2} \rightarrow \text{درجه ۲}$$

لے میں مائنس ہم درجہ ۲ خواہد بود.

رشد توابع

big big little
O, Ω, Θ, o, ω

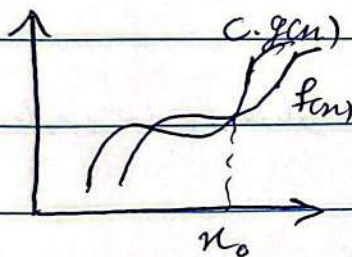
توابع زمان اجرا مثبت و صعودی اند:

$f(n) \in O(g(n))$ → معادل

تعریف:

$$\exists c > 0, n_0 \geq 0 : \forall n \geq n_0 ; f(n) \leq c \cdot g(n)$$

(حد بالایی رشد)



در واقع پس از یک مقدار رشد یانرخ افزایشی
g از f بیشتر است.

رشد توابع چند جمله ای به درجه وابسته است.

یعنی $O(g(n))$ برای توابع چند جمله ای، مجموعه ای از توابع است که رشد معادل یا بیشتر از f دارد، یعنی هم درجه یا بالاتر.

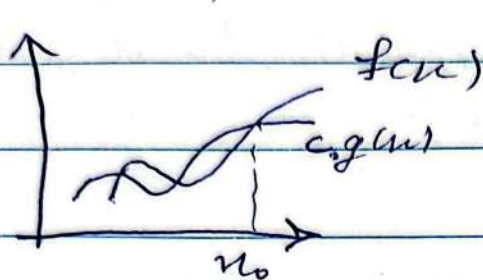
در وقوع 0 مکتب زمان $T(n)$ از g بدتر نیست، $\lim_{n \rightarrow \infty} \frac{g}{f} < \infty$ $C \neq 0$

$$f(n) = 2n + 10 \in O(n), O(n^2) \not\subset O(\sqrt{n})$$

آذین مهر

اگر $f(n) \in \Omega(g(n))$

$$\exists c > 0, n_0 > 0 : \forall n \geq n_0; f(n) \geq c g(n)$$



(حد پایین)
 دقتاً، تابع f بیشتر یا مساوی
 تابع g باشد یعنی $f(n) \geq c g(n)$

پس اگر $f \in O(g)$ داریم $g \in \Omega(f)$

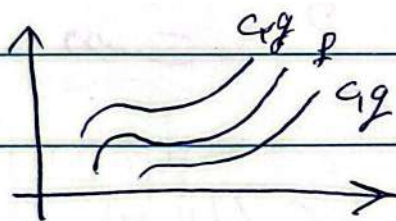
$$n + \log n \in \Omega(n), \Omega(\sqrt{n}) \notin \Omega(n)$$

در ختم مسئله، نشان می‌دهد $\lim_{n \rightarrow \infty} \frac{f}{g} = c \neq 0$

مثلاً اثبات می‌شود یک مسئله $\Omega(n^5)$ است یعنی زودتر از n^5 حل نمی‌شود.

اگر $f(n) \in O(g(n))$

$$\exists c > 0, n_0 > 0 : \forall n \geq n_0; c g \leq f \leq C_2 g$$



یعنی دو تابع هم رشد
 هستند.

$$n + \log n \in O(n) \quad \{f \in O(n^2)\}$$

برای تعریف اثبات می‌کنیم $T(n)$ چه مقدار دارد $\lim_{n \rightarrow \infty} \frac{f}{g} = c$

$$n + \log n \stackrel{?}{\leq} c n \rightarrow (c-1)n - \log n \stackrel{?}{\geq} 0$$

$$c > 1 : n - \log n \geq 0 \rightarrow n \geq \frac{\log n}{c-1}$$

پس $c = 2, n_0 = 4$ وجود دارد، تعریف برقرار است. $\log n \geq \frac{\log n}{c-1}$

$$f \in O(g(n)) \Leftrightarrow g \in O(f(n)) \Leftrightarrow f \in \Omega(g(n))$$

اذین

$f(n) \in o(g(n))$ اگر

$$\forall c > 0, \exists n_0 > 0, \forall n \geq n_0, f(n) \leq c g(n)$$

و $g(n)$ باید بیشتر و نامساوی باشد (محدود) و $f(n)$ باید کمتر باشد

$$\lim_{n \rightarrow \infty} \frac{f}{g} = 0$$

$f(n) \in \omega(g(n))$ اگر

$$\forall c > 0, \exists n_0 > 0, \forall n \geq n_0, f(n) > c g(n)$$

و $f(n)$ باید کمتر و نامساوی باشد (محدود) و $g(n)$ باید بیشتر باشد

$$\lim_{n \rightarrow \infty} \frac{f}{g} = \infty$$

$$n^2 + 1 \in o(n^3), o(n^3) \subseteq \omega(n^2)$$

از Ω یا Θ (نویس) هیچ اطلاعی نداریم: $n^{1+\sin n}$

درست؟ $f(n) + o(f(n)) \in O(f(n))$ ؟

$f(n)$ و $h(n)$ کمتر از $f(n)$

$$\left\{ \begin{array}{l} f \in o(g) \\ f \in O(g) \\ f \in \omega(g) \end{array} \right\} \rightarrow \text{رابطه‌های (الهام) شکل توابع میوه \rightarrow غایب}$$

$$f \in \Omega(g) \& f \in O(g) \Rightarrow f \in \Theta(g)$$

$$f \in O(g) \& g \in O(f) \Rightarrow f \in \Theta(g)$$

$$f \in O(f), f \in \Omega(f)$$

$$f \in O(g) \Leftrightarrow g \in \Omega(f)$$

آزین

ساختار داده - Data Structures

DS های ساده: آرایه (array)، صف (Queue)، پشته (stack)

آرایه: مجموعه ای از خانه های فیزیکی متوالی از نظر فیزیکی که هم نوع اند. (حتی هم باز بودن برای دسترسی اندیس لازم است.)

آدرس شروع آرایه $\rightarrow A$ // `int A[10];`

$A[0] \rightarrow$ خانه اول $A[2] \rightarrow \text{address } A[0] + 2 \times \text{sizeof}(\text{type})$

مزیت آرایه
 $A[i] \rightarrow A + i \times \text{sizeof}(\text{type})$
 دسترسی به آرایه در $O(1)$ انجام می شود.

اما در زمان کامپایل ساینر آرایه ثابت است و تغییر ساینر باید به صورت دستی انجام شود. افزودن حذف کردن باید با `memset` یا `memset` که دانشی، کسی کردن دستی انجام می شود.

آرایه چند بعدی

		a			
b	i \ j	00	01	02	03
	10				
	20				
	30				
	40				

آرایه چند بعدی هم دهافقا نیست سرهم نیست و

طرزهای بسته دهافقا حیده می شود.

$$A[i][j] = A + i \times a \times \text{sizeof}(\text{type}) + j \times \text{sizeof}(\text{type})$$

در واقع برای آرایه n بعدی، داشتن a بعد برای کار کردن با آرایه لازم

است ولی داشتن بعد آخر لازم نیست.

$$A[i][j][k] = A + i \times a \times \text{sizeof}(\text{type}) + j \times b \times \text{sizeof}(\text{type}) + k \times \text{sizeof}(\text{type})$$

اول

برای ساینر یا س دابل `array` یا `func` بعد اول لازم نیست.

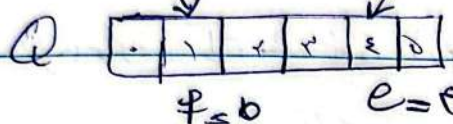
آزین

FIFO/LIFO

Queue

چنین اطلاعات به صورت که اولین داده انجام شده اولین بار خارج شود.
 صف جدید queue array یا linkedlist پیاده می شود.

Front Pointer به سمت چپ
 End Pointer به سمت راست



front == end

end > front

(end را یک پیکان از ابتدا به آخر دقت می کنیم)

method EnQueue (x) $\rightarrow O(1)$

$Q[end] = x$ $\rightarrow$ if end $\leq$ max

end ++ $\rightarrow$ else print ('queue is full')

صف یکبار پر می شود. اگر یکبار آرایه به انتها برسد دیگر نمی توان از صف استفاده کرد

method de Queue (x) $\rightarrow O(1)$

if end > front?

$x \leftarrow Q[front]$

front ++

return x

else print ('Q is empty')

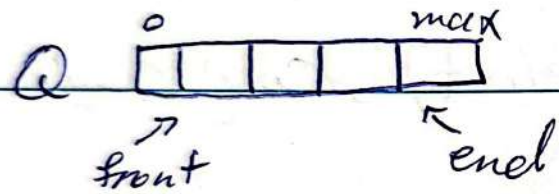
method isFull () $\rightarrow O(1)$

return end > max

method isEmpty $\rightarrow O(1)$

return front == end

آذین



آرایه بک (صف) صف چند بار صاف
 در آن خواهیم نوشت و end و max به آن فضای

ابتدای array آزاد بود از آن استفاده شود: $end = max + 1$

$is_empty()$

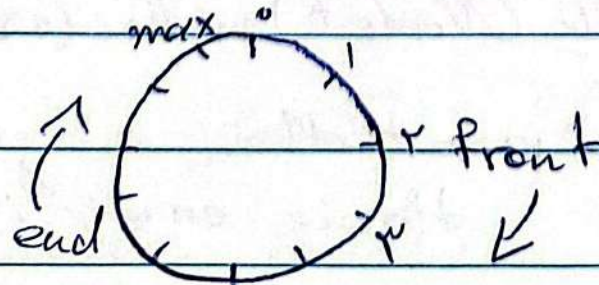
$return\ end == front;$

با این شرط
 خانه $front$ خالی

$is_full()$

$return\ (end + 1) \% size == front;$

end و max
 خالی می گذاریم



آزمایش

enqueue (x):

$Q[rear] = x$

$rear = (rear + 1) \text{ mod } size$

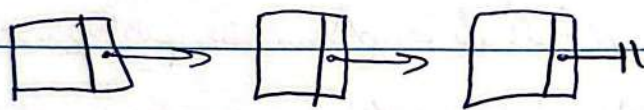
dequeue ():

$x = Q[front]$

$front = (front + 1) \text{ mod } size$

return x

Q مبتنی بر array یا لیست پیوندی (Linked-list) DS میسر است.



هر خانه حافظه برای یک عضو داده

در نظر گرفته می شود. آدرس عضو بعدی (next) را ذخیره می کند.

در هر ترانه یک طرفه و یک طرفه یا حلقوی باشد.

isEmpty ()

$return front == Null$

Add ()

```
struct Node {  
    int item  
    Node* next  
}
```

$Node* P = new Node$ یا $(Node*) malloc(sizeof(Node))$

$P->item = item$ → if $p == Null$

$P->next = front$

$front = P$

throw error / return -1

آزمین

del()

if Front == Null

return -1; or throw error

Node * P = Front

Front = Front → next

Free(P)

return 0;

Find (int item) → Find by value

Node * p = Front;

while (p != Null)

if p->item == item

return p;

p = p->next

return p; ↳ return Null;

class LinkedList

{ private struct Node;

private Node* First;

public ~~front()~~, add(), del(), isEmpty()

private Find(),

آزیر

اداره linkedlist - یک لیست

Adel (P) از اشارت $\rightarrow O(n)$

$q = \text{Front}$

if $q == \text{Null}$

$\text{Front} = P$

$p \rightarrow \text{next} = \text{Null}$

else

while $q \rightarrow \text{next} != \text{Null}$

$q = q \rightarrow \text{next}$

$q \rightarrow \text{next} = P$

$P \rightarrow \text{next} = \text{Null}$

Add (P) $\rightarrow$ در انتهای لیست

if $\text{end} == \text{Null}$ // لیست خالی

$\text{Front} = \text{end} = P$

$P \rightarrow \text{next} = \text{Null}$

else

$\text{end} \rightarrow \text{next} = P$

$P \rightarrow \text{next} = \text{Null}$

$\text{end} = P$

del() while

if end == Null,

return;

elif front == end

front = end = Null

else

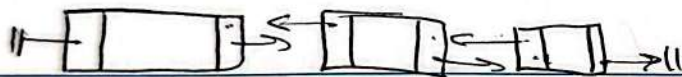
q = front

while q->next != Null

q = q->next

end = q

q->next



لیست - دیکر

Add Front(p)

p->next = front

front->prev = NULL

p->prev = NULL

front = p

if front == Null

front = p

p->next = Null

p->prev = Null

del Front()

if front == Null

return

elif front->next == Null

front = Null

else

front = front->next

front->prev = Null

آذین

delLast ()

if end == null

front = end = P

else

end->next = P

P->prev = end

next = null

end = P

delLast ()

if end == null

return

else if front == end

front = end = null

else

end = end->prev

end->next = null

delNode (q) (جمله)

q' (جمله) -> next = q -> next

آمین

Add Node After (P) کے لیے

$P \rightarrow next \rightarrow Q \rightarrow next$

$Q \rightarrow next \rightarrow P$

Add Node Before (Q) کے لیے

$(Q \rightarrow prev) \rightarrow next \rightarrow Q \rightarrow next$

$(Q \rightarrow next) \rightarrow prev \rightarrow Q \rightarrow prev$

Add After (Q) کے لیے

$P \rightarrow next \rightarrow Q \rightarrow next$

$(Q \rightarrow next) \rightarrow prev \rightarrow P$

$P \rightarrow prev \rightarrow Q$

$Q \rightarrow next \rightarrow P$

یہ سب دیکھ کر حلقہ بنانا ہے۔

آزیم

مقایسه آرایه و لیست

- آرایه Fixed ولی لیست dynamic ساخته است.

- دسترسی آرایه $O(1)$ و لیست $O(n)$ است.

- جستجوی هر دو $O(n)$ است. (Linear search)

- اگر مرتب باشد در آرایه $O(\log n)$ است. (Binary search)

- ولی در لیست همان $O(n)$ است.

- حذف و افزودن در لیست $O(1)$ است. ولی در لیست آرایه ممکن

است shift یا swap نیاز باشد. (shift $\rightarrow O(n)$)

- لیست پیوندی حافظه بیشتری برای pointer ها نیاز دارد.

پیاده سازی Queue با linkedlist

حذف با آرایه ظرفیت محدود دارد ولی در لیست محدودیتی ندارد.

لیست در لیست یک طرفه بهتر است حذف از front، افزودن به end انجام شود.

enqueue(P)

front = NULL

end & next = P

end = end->next

enqueue()

P = front

front = front->next

آذین

LIFO $\rightarrow$ Stack $\rightarrow$ $\sum_i$

۱- مانند برگشت از فراخوانش توابع

main $\xrightarrow{\text{call}}$ f $\xrightarrow{\text{call}}$ g $\xrightarrow{\text{call}}$ h

$\xleftarrow{\text{return}}$ $\xleftarrow{\text{return}}$ $\xleftarrow{\text{return}}$

یعنی سسٹم عامل باہر کا function call کے انجام میں خود آکر return
کا در stack اضافہ میں آئے گا۔

class Stack

```
int S[max];
```

int endl;

publico

```
push(int x){
```

if (this == isFull()) return -1;

$S[+ + \text{end}] = 2$

```
return 0; }
```

pop(int x) {

if (this is Empty()) return -1;

$X \in S[\text{end}]$;

encl - - ; $\rightarrow$

با حنف سن حنا

return X ;

TopCS in

13 Empty of

انجام شد.

return end = z - 1; }

is full $\rightarrow$ end $\rightarrow$ max $\downarrow$

آذین

is Empty () { return end == NULL; }

push () {

Node *p = new Node;

p->item = x;

p->next = end;

end = p;

return;

pop () {

if (is Empty ()) return NULL;

Node *p = end;

~~end->next~~ end = end->next;

X is Problem

free/delete p;

return x;

Arithmetic exp evaluated by stack

$$60 + 60 \times 2 - 5 \times 5 \div (2 + 2)$$

اولویت ضرب و تقسیم اولویت پرانتز

عملگر بین عملوند ها infix: $a + b$ سلفرم
 عملگر بعد از عملوندها postfix: $ab +$ علامت
 عملگر قبل از عملوندها postfix: $+ ab$ بیس

postfix در کد کامپیوتر آسان تر است چون عملگر را تشخیص می‌دهد و نیازی نیست که نیاز به پرانتز ندارد.

$a + b \times c$ $\begin{cases} (a + b) \times c & ab + c \times \\ a + (b \times c) & a \ b \times c + \end{cases}$

ابتدا عبارت Infix به Postfix توسط stack
 لیست می‌شود و سپس به واسطه stack کامپایل می‌شود.

کامپایلر stack را تبدیل postfix $\rightarrow$ infix

هر حرف که در stack قرار می‌گیرد $a + b \times (c \div (d + e)) \times f$

تغییر عملگرها $\rightarrow$ stack	خروجی	عملیات
$+$	a	a
$+ \times$	a, b	$+$
$+ \times ($	a, b	$\times$
$+ \times (\div$	a, b, c	$($
$+ \times (\div +$	a, b, c	$\div$
$+ \times (\div +)$	a, b, c	$+$

آزمایش

ورودی	stack	خروجی
d	+ x (/ C	ab cd
+	+ x C / C +	ab cd
e	+ x C / C +	ab cd e
)	+ x (/	ab cd e +
)	+ x	ab cd e + /
X		حالی که هم اولویت هستند پس اولویت بیشتری دارد.
	+ x	ab cd e + / x

(وقتی که stack می توان push انجام داد که اولویت بالاتر از top داشته باشد)

f	+ x	ab cd e + / x f
		ab cd e + / x f + x

در اینجا باقی مانده pop می شود.

→ الگوریتم: ورودی tokenize می شود و هر بار

۱. اگر عملوند باشد در خروجی اضافه می شود

۲. اگر عملگر باشد فقط وقتی اضافه می شود که از top اولویت بیشتری

داشته باشد. وگرنه تا وقتی که اولویت از top کمتر است از stack

pop می کنیم و در رشته می نویسیم. (اگر یک عملگر باشد آن که از قبل آمده اولویت دارد)

۳. اگر (باشد در stack آن را push می کنیم. (یعنی توانسته بود آن را به stack اضافه کند)

۴. اگر (باشد آن قدر pop می شود تا () (یا تا وقتی که نیافت)

۵. تعیین عملگرها (مثل +, -, *, /) و توابع tokenize انجام می شود.

آزاد

حاسب جواب postfix با stack

$$10 + 2 \times ((3 + 4) - 3) + 8 / 2 \times 2$$

postfix $\rightarrow 10 \ 2 \ 3 \ 4 + 3 - \times + 8 \ 2 / 2 \times +$



سیس برای محاسب از چپ راست می‌رویم و هر جا به عملگر رسیدیم
دو عدد از top پشته pop می‌کنیم و نتیجه را push می‌کنیم

فرای stack				
10	2	3	4	(+)
10	2	7	3	(-)
10	2	4		(x)
10	8			(+)
18	8	2		(/)
18	4	2		(x)
18	8			(+)
۲۶				

گراف ها (graph)

$$G = (V, E)$$

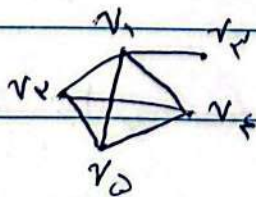
مجموعه رئوس

مجموعه یالها

نوع مرتب

نوع مرتب د جهت دار

مجموعه یالها جهت دار



$$V = \{v_1, v_2, v_3, v_4, v_5\}$$

$$E = \{(v_1, v_2), (v_1, v_3), (v_1, v_4), (v_1, v_5), (v_2, v_3), (v_3, v_4), (v_4, v_5), (v_5, v_1)\}$$

آزمایش

$v_1 \rightarrow v_2$
 $\rightarrow v_3$

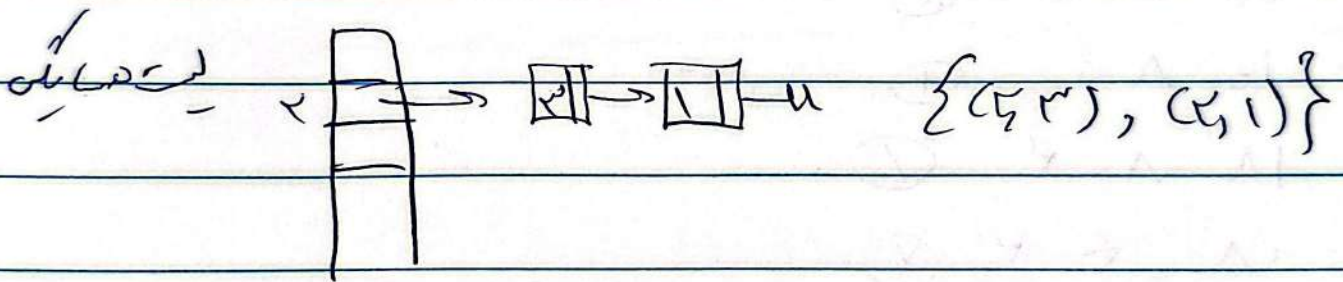
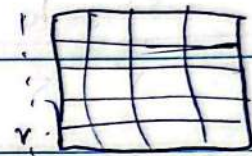
رنگ سبب
 $E = \{(1,2), (1,3)\}$

گراف
 جهت دار

گراف وزن دار (weighted) : هر یال عدد به عنوان وزن نسبت به دهیم.

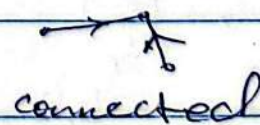
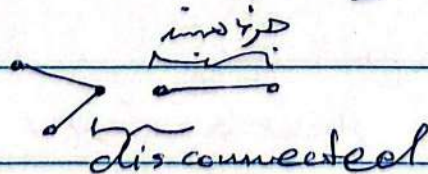
DS for graph $\rightarrow$ matrix $\rightarrow$ ماتریس
 $\rightarrow$ list $\rightarrow$ لیست

$A[i][j] = \begin{cases} 1 & \text{یال } i \rightarrow j \\ 0 & \text{بدون یال } i \rightarrow j \end{cases}$



graph, adjacent list, adjacent matrix

گراف
 ماتریس همبندی: هر v_1, v_2 یک مسیر وجود دارد. (صرف نظر جهت)

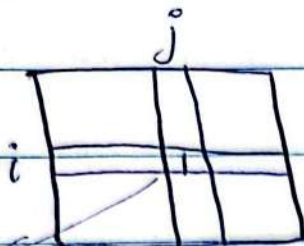


می توانیم جزء همبندی (connected component) وجود داشته باشد.

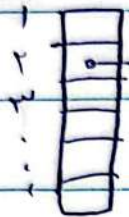
آزمایش

$\Theta(n^2)$ کا وقت

(۱) بر روی محور O



$(i, j) \leq 1$ (مقدار اول)



$\Theta(n+1)$

θ (degree) $\rightarrow \theta$ (rad)

اگر بدل جهت باشد ما نیز در حساب این متقابل است. بدون جهت
را در واقع از هر دو طرف ذخیره می کنیم.

تکثر E نسبت به γ در $\gamma = 1$ به صورت زیر است:

$$|E| \leq \frac{\gamma(1-\gamma)}{\gamma} = 1 - \gamma$$

سے ہیں اگر تعدادِ مال کم جائے لیکن بہتر است مگر نہ جائز کہ بہتر است۔

$\text{degree}(v)$: تعداد یال‌های هم‌مسیر به یک رأس

لے در کمرلف جہت دار برای in و out درجہ جداگانہ تعریف مرکب

$$\text{in-degree}(v) + \text{out-degree}(v) = \text{degree}(v)$$

فصل ۷: رئیس هاست که بیکی مال ۷ میل است. (outgoing در رجب دارها)

• $(v, u) \in E$ إذا كان v متجاورا لـ u

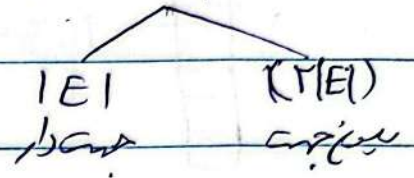


کرایہ کامل: دہائیہ مال ہمارے ملک (Complete)

آذین مہر

۱. بعضی اعمال در لیست سریع تر است. مثل $in-degree$ و $out-degree$ برای هر رأس.

۲. $degree(v)$: لیست { درجه }
 $O(n)$: لیست { پهنایی }
 $O(n)$: لیست { یک رأس }

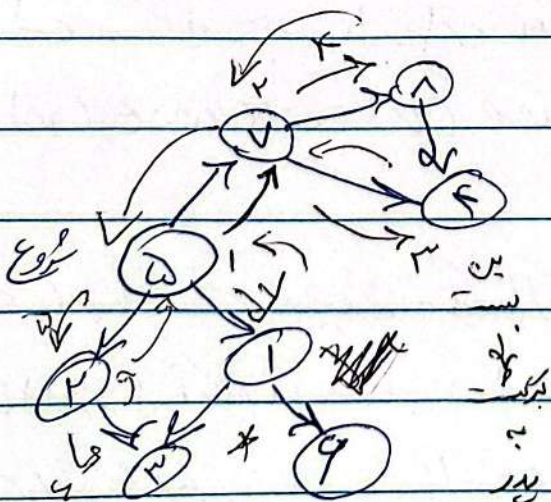


گراف نازک (sparse) که پهنایی دارد پهنایی مناسب است.
 به ازای n کمتر از n یا $n \log n$

پهنایی در گراف: حرکت کردن بر روی رأس و پهنایی در گراف.

۳. الگوریتم‌های معروف: DFS (Depth First search) و BFS (Breadth First search) به ترتیب

DFS از اولین سر به سر اول DFS به صورت عمیق و سطحی حرکت می‌کند.



۱. از یک رأس شروع می‌کنیم.

۲. به همسایه $visit$ نشده می‌رویم تا به بن‌بست.

۳. به عقب برمی‌گردیم تا همسایه $unvisit$ شده پیدا کنیم.

۴. اگر رأس می‌رویم. (در رأس شروع می‌کنیم و $visit$ می‌کنیم).

۵. پس کار را تا آخر می‌کنیم تا تمام گراف را $visit$ کنیم.

۶. اگر رأس $unvisit$ شده از آن عبور می‌کنیم.

شروع می‌کنیم.

پس الگوریتم به ۲ نمی‌رسد.
 ۱. مقولان عبارتند از DFS و BFS .

آذینه

شرط خانه DFS: در خانه کج با یک مربع $visited$ و $unvisited$ است که 0 یا 1

برای لین الگوریتم از $recursion$ و $stack$ استفاده می کنیم.

DFS(G, s):

فایده دیدن حالت

حالت کل FE

1. $visited[s] = True; \rightarrow C_1$

2. For w in neighbors(s) do: $\rightarrow O(n_1 + degree) = O(|E| + |V|)$

3. if $visited[w] == False: \rightarrow che: C_2$

~~$visited[w] = True;$~~

4. DFS(G, w)

1. che list $\rightarrow matrix$

2. $\rightarrow O(|E| + |V|) \leq O(|V| + |V|) = O(|V|)$

3. $\rightarrow C_2(degree(s) + 1)$

4. $\rightarrow degree(s)$

5. $\rightarrow T(V_1) + T(V_2) + \dots + T(V_n) \rightarrow O(V)$ (رابطه بازگشت)

هر یک در DFS به بار یک می شود

برای تحلیل زمان هم به صورت الگوریتم و هم به صورت معادله ای می توان عمل کرد. در اینجا به طور کلی گفتیم که تحلیل کردیم.

آذین مهر

در هر دو حالت DFS یک زمان خطی نیاز دارد

$O(V^2)$ زمان و V^2 اندازه ورودی $\rightarrow$ matrix

$O(V+E)$ زمان و $V+E$ اندازه ورودی $\rightarrow$ list

درم الکوریتم بر حسب اندازه ورودی بیان می شود نه V یا E
تا به زمان اجرا نماند بر حسب بیان ورودی (V) بیان می شود.

وقتی دو الگوریتم متفاوت حذف کند لزوماً سرعت یکسانی ندارند،
در اینجا list معمولاً سریع تر است.

wrapper برابر غیر فایده ایست

$DFS_Wrapper(G, s)$

$DFS(G, s)$

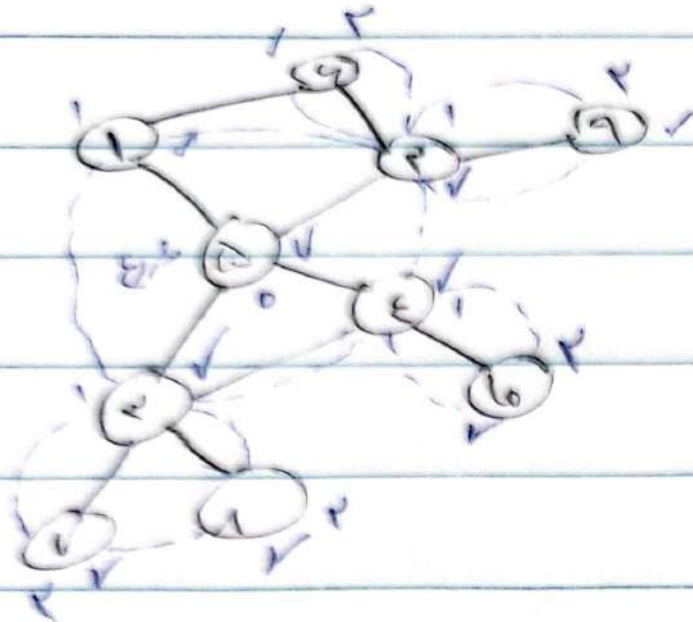
for i in $vertices$:

if i is $False$:

$DFS(G, i)$

تحلیل زمانی همان لیست است، چون نسبت به سایر ورودی ها مقایسه می کنند.

الگوریتم BFS



از node label شروع می‌کنیم و label 0

و اگر label 0 منگونی، سپس از label 1

همین کار را تکرار می‌کنیم.

حداکثر label برابر عمق گراف است.

یا نزدیکترین کوتاه‌ترین مسیر.

BFS

یک صف گشوده و هر بار عناصر هار را از آن برداشتی و
به صف اضافه می کنی:

BFS (G, s),

Array

Queue Q; visited[]; label[]; label[s] = 0; visited[s] = true;
while !Q.is Empty:

u = Q.dequeue();

visited[u] = true;

label[u] = l

↑
order
شماره

for w in neighbors[u]:

if visited[w] == 0:

visited[w] = 1

label[w] = label[u] + 1

Q.enqueue(w)

dequeue, enqueue, l, v

کلیه یابی:

$T_{BFS} \in O(|V| + \sum \text{degree}(u)) = O(|V| + |E|)$
 $\uparrow$
 $|V|^2$ $|E|^2$

هر دو جنبه سازگاری حفظ است.

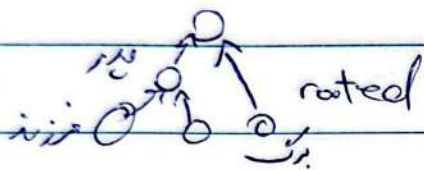
آذین

Linked list ها طی گفته ترین مسیر از راس به راس و به سمت Label
راشته می شود.

درخت: گراف جهت بدو جهت دور

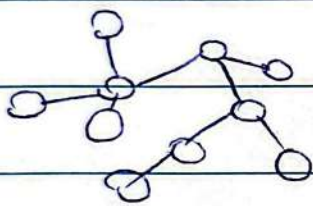
در درخت نمی توان از یک راس به همان برگشت.

یعنی از لیست و ماتریس همبستگی می توان بهر دقتی درخت استخراج کرد.



درخت می تواند جهت داشته باشد یا نه.

directed tree (جهت دار) و undirected tree (جهت ندارد) (جهت دارد یا ندارد)



درخت آزاد (free tree): هیچ جهت ندارد.

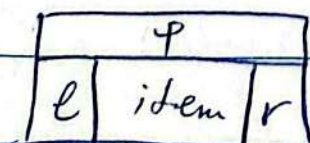
(گراف بدون جهت ساده)

در درخت ریشه دار اگر $(v, w) \in E$ و $(w, v) \notin E$ (فرزند است)

که (node) بدون فرزند برگ است.

اگر هر پدر حداکثر دو فرزند داشته باشد درخت دودویی است. (Binary tree)

درخت ریشه دار می تواند: هر پدر حداکثر یک فرزند دارد.



نحوه پیاده سازی DS برای BRT:

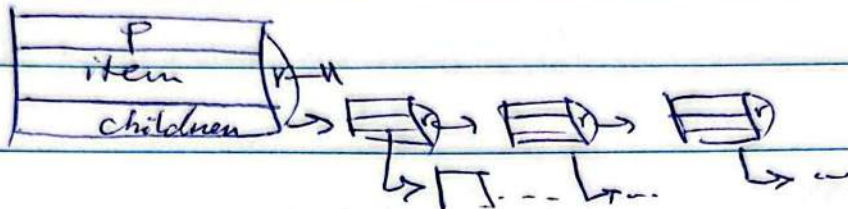
Parent l: left r: right

آزمون

P	
item	
k_1	k_n

برای رخت خانه
از روشی مناسب می توان استفاده کرد:

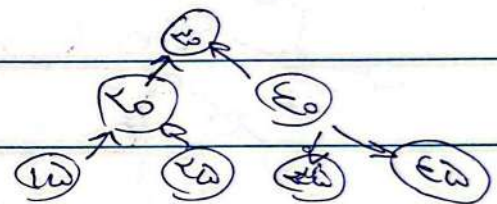
می توان خود خانه را اشاره کرد و به صورت linked-list تعریف کرد:



BST (Binary search tree):

مقدار کلید فرزند راست بزرگتر یا مساوی پدر است. (همیشه بزرگتر)

$$\begin{cases} x.l.key \leq x.key \\ x.r.key \geq x.key \end{cases}$$



عمق ۲، ارتفاع ۳

ارتفاع (height) یک گره:

طول طولانی ترین مسیر از برگ به گره مورد نظر
(طول مسیر تعداد نودها)

ارتفاع رخت ۳ ارتفاع رخت درخت.

عمق گره (Depth): طول مسیر از گره به رخت

پس در بیشترین حالت چه $O(D)$ است
که ارتفاع رخت

آزمین

لیس برابر آرایه ریج sorted می توان از الگوریتم BST استفاده کرد.
که $O(\log n)$ است. حذف و ریج BT هم $O(\log n)$ است.

استخراج اعداد sort شده از BT

یہاں سے دیکھ جائیں

pre-order

in-order

post-order

دائیں سے بائیں

بائیں سے دائیں

کبھی سے آخر

یہ پس بکھر اسفنج order سے جس میں order کے معنی ہیں کہ وہاں لکھ لکھ رہے ہیں

In-Order Tree walk (2):

if $x \neq \text{Null}$: $(\neg(x \in \text{Null}))$

In Order Tree-walk (n.c)

```
print(m)
```

In Order Tree Walk (iur)

$$T(n) \leq C_1 + T(k) + C_2 + T(n-k-1) + C_3$$

$$\rightarrow T(n) = T(k) + T(n-k-1) + O(n)$$

عکس: $T(n) = O(n)$ چون به ازای هر n هزینه ثابت است. print

فرض: $T(0) \leq c$ $T(n) \leq (c+d)n + c$ $\forall n \geq 0$

آذین مہر

$$n \leq 0 \rightarrow T(0) = c \leq (c+d) \cdot (0) + c \leq c \quad \text{پایه (حالت گذار)}$$

چون دوتا بازگشت وجود دارد از استقرا قویتر می‌رویم.
یعنی فرض می‌کنیم برابر هر مقدار کمتر از n برقرار باشد و برای n
نشان می‌دهیم:

$$k < n, \quad n-k-1 < n$$

$$\text{فرض} \begin{cases} T(k) \leq (c+d)k + c \\ T(n-k-1) \leq (c+d)(n-k-1) + c \end{cases}$$

$$\rightarrow T(n) = T(k) + T(n-k-1) + d$$

$$\begin{aligned} \rightarrow T(n) &\leq [(c+d)k + c] + [(c+d)(n-k-1) + c] + d \\ &\leq (c+d)n + c \end{aligned}$$

توان از لعل $(an+b)$ تعیین کرد

شرایط a و b را پیدا کرد

$$1. \quad T(0) = c \stackrel{?}{\leq} b \rightarrow \text{شرط: } b \geq c$$

$$2. \quad T(n) = T(k) + T(n-k-1) + d \leq an + b + (b + d - a)$$

$$\rightarrow an + b + (b + d - a) \stackrel{?}{\leq} an + b$$

$$\rightarrow b + d - a \leq 0 \rightarrow \text{شرط: } a \geq b + d \geq c + d$$

Tree Search (x , k):

if $x == \text{Null}$ or $x.\text{key} == k$

return x ;

if $k < x.\text{key}$

return Tree Search ($x.l, k$);

else

return Tree Search ($x.r, k$);

زمانی: $O(h)$ (ارتفاع درخت)

$$T(n) = C_1 + T(n) + \max\{T(h), T(n-k-1)\}$$

worst case: $T(n) = C_1 + T(n-1) \in O(n)$

Balance: $T(n) \in O(\log n)$

height based: $T(h) = C + T(h-1) \rightarrow T(h) \in O(h)$

$\max\{T(h), T(h)\}$ $\rightarrow$ worst case

$\rightarrow$ search, minimum, maximum, Insert

BST

delete, successor, predecessor, ...

Iterative Tree Search (x , k)

(غیر بازگشتی)

while $x \neq \text{Null}$ and $k \neq x.\text{key}$

if $k < x.\text{key}$: $x = x.l$

else $x = x.r \rightarrow T(h) \in O(\text{height})$

برای گرفتن predecessor در یک B+ tree، left، right، min، max را می‌گیریم.

Insert (T, z)

y = Null

x = T.root

while x ≠ Null

y = x

if z.key < x.key : x = x.left

else: x = x.right

z.p = y

if y == Null

T.root = z

else if z.key < y.key

y.left = z

else: y.right = z

از ریشه پایین می‌رویم تا به

ایره که Null برسد.

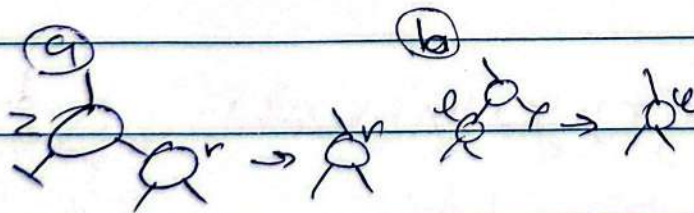
مسابقه با کلید

search

فرزند چپ یا راست

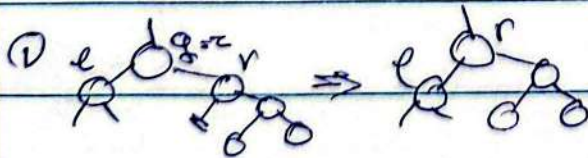
بدا می‌کنیم

حذف delete(z)



(a) فرزند چپ ندارد

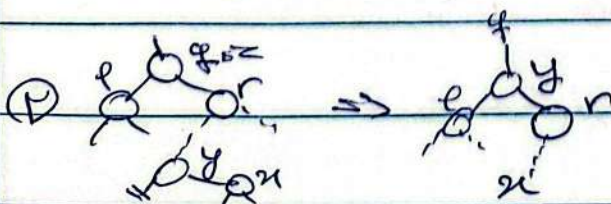
(b) فرزند راست ندارد، پس چپ را برده



(c) فرزند راست چپ ندارد

(d) فرزند راست succ است.

(e) فرزند راست succ نیست.



آزمین

Transplant(^uT, ^vu, ^vr)

if $u.p == \text{Null}$

$T.\text{root} = v$

else if $u == u.p.\text{left}$

$u.p.\text{left} = v$

else $u.p.\text{right} = v$

if $v \neq \text{Null}$: $v.p = u.p$

جایگزینی r

در u جایگزینی

Tree Delete(T, z)

if $z.\text{left} == \text{Null}$

transplant(T, z, z.right)

else if $z.\text{right} == \text{Null}$

transplant(T, z, z.left)

else

$y = \text{Minimum}(z.\text{right})$

if $y.p \neq z$

transplant(T, y, y.right)

$y.\text{right} = z.\text{right}$

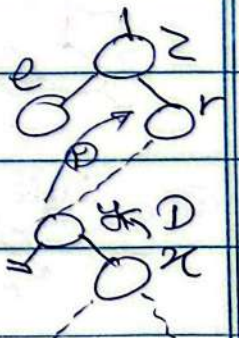
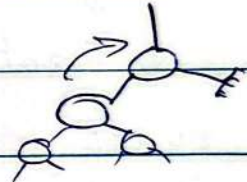
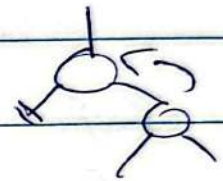
$y.\text{right}.p = y$

transplant(T, z, y)

$y.\text{left} = z.\text{left}$

$y.\text{left}.p = y$

آذین



تحلیلی زمانه: $T_{ins} O(n) \rightarrow$ Insertion (...)

به خاطر mem گشتی $\rightarrow O(n) \rightarrow$ Delete

ساخت BST متوازن (red-black tree)

۱- هان مثل BST اما هر گره رنگ قرمز یا سیاه دارد. قوانین:

۱- هر گره دقیقاً قرمز است یا سیاه.

۲- ریشه سیاه است.

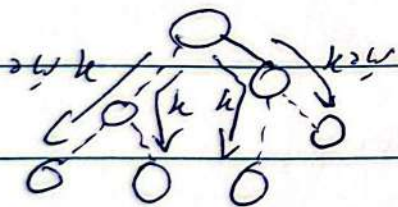
۳- گره Null (که در اینجا به معنای برگ singleton است) سیاه است.

۴- اگر گره امر قفز باشد، هر دو فرزند آن سیاه است.

۵- (یعنی دو گره نیست هم می تواند سیاه باشد و می تواند قرمز باشد)

۶- بلر هر گره، هم مسیرهای سیاه از گره تا برگ هر زیرین شامل تعداد

یکسان از گره سیاه است. به ویژگی اصلی RBT



چون در تمام مسیرها یک گره سیاه است و

فرز نیست هم نیست، می توان گفت هیچ

مسیر از ریشه برابر دیگر نیست.

اثبات می شود وقتی حد اختلاف ارتفاع از برابر بیشتر شود الگوریتم ها نگهداری می کنند

ارتفاع گره (black height)، تعداد گره های سیاه از ریشه تا برگ ها $bh(n) = k$

آذینه

قضیه (Chemmer): یک RBT با n گره internal (گره غیر $leaf$) دارای ارتفاع حداکثر $\log(n+1)$ است.

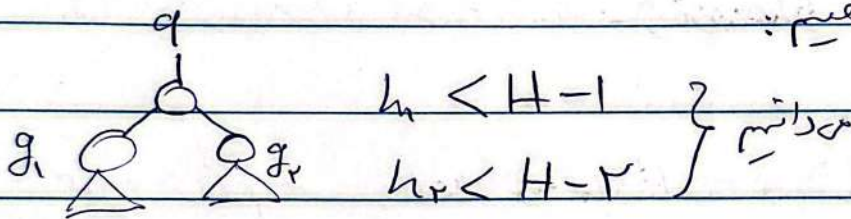
لح (اثبات): D ادعا: هر زیر درخت از q (بار q) حداقل 1 - $bh(q)$ عدد گره داخلی دارد.

ادعا 1 با استقرار قوی ثابت می‌کنیم:

$$bh(x) = 0 \rightarrow (x \text{ یه}) \rightarrow bh(x) = 1$$

تعداد گره داخلی $\rightarrow$ $1 \leq 1 - 1 = 0$

حال فرض کنیم ادعا برای تمام درخت‌ها با ارتفاع کمتر از h درست است و بار h را به دست می‌دهیم:



$$\left. \begin{aligned} \text{internal}(q_l) &\geq 2^{bh(q_l)} - 1 \\ \text{internal}(q_r) &\geq 2^{bh(q_r)} - 1 \end{aligned} \right\} \Rightarrow$$

$$bh(q_l) = bh(x) \text{ or } bh(x) - 1 \rightarrow bh(q_l) \geq bh(x) - 1$$

لحظه آخر با 1

$$\rightarrow \text{internal}(q_l) + \text{internal}(q_r) + 1 \geq 2^{bh(q_l)} + 2^{bh(q_r)} + 1$$

آزمایه

اثبات: یک اصل با اعداد گفته شد:

$$n \text{ nodes} \geq r^{bh(n)} - 1$$

چون در هر سطح حداقل یک گره وجود دارد.

$$\Rightarrow bh(n) \geq h(n)/r \Rightarrow n \geq r^{bh(n)} - 1 \geq r^{h(\text{root})/r} - 1$$

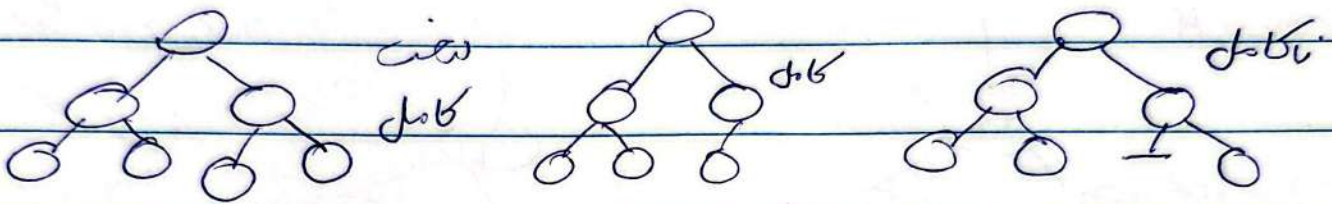
$$\Rightarrow r^{h(\text{root})/r} \leq n+1 \Rightarrow h(\text{root}) \leq \frac{r}{r-1} \log_r (n+1)$$

Augmented DS

نقوت RBT با بعضی ندهاها اطلاعاتی که کاربرد گسترده‌تری دارد.

Heap Tree

درخت (معمولاً باینری) و کامل (هر نود در درخت دارای به جز سطح پایانی)



باید فقط از سمت راست خالی باشد.

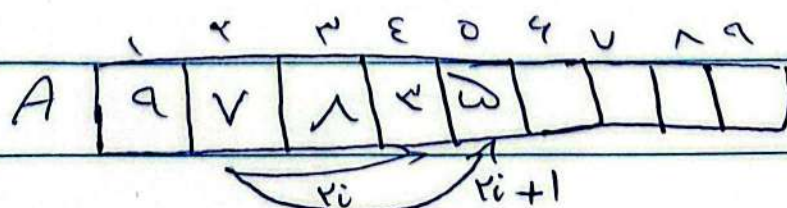
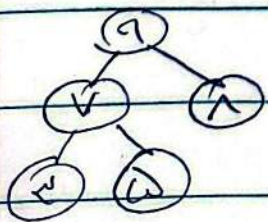
Max Heap: $\rightarrow$ گره بزرگتر $\leftarrow$ گره کوچکتر

Min Heap: $\rightarrow$ گره بزرگتر $\leftarrow$ گره کوچکتر

برای حذف و آیدیت، آخرین بزرگ‌تر را به جای نود حذف برد
گذاشته و در بار بین نود فعلی و نود فرزند max را در محل
نقدی می‌گذاریم.

مناسب باینری tree کامل
که راست از آن‌ها خالی باشد.

پایه سازی درخت Heap آرایه

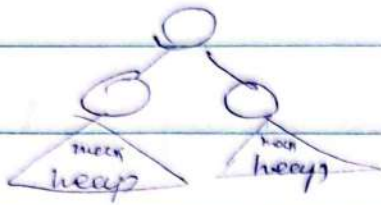


length(A) = 9

size(A) = 5
heapsize

آزین

$$\text{left_child}(a) = 2i_a, \text{ right_child}(a) = 2i_a + 1, \text{ parent}(a) = \lfloor \frac{i_a}{2} \rfloor$$



heap درخت

این حالت دارد: درخت باینری کامل

که زیر درخت راست هیچ آن max heap

است ولی کل درخت نه. می خواهم آن max heap کنیم

درست

$$T(n) = O(n) + T\left(\frac{n-1}{2}\right)$$

Max-Heapify(A, i) ↗

$l = 2i, r = 2i + 1$

if $l \leq \text{heap_size}[A]$ and $A[l] > A[i]$

largest = l

else largest = i

if $r \leq \text{heap_size}[A]$ and $A[r] > A[\text{largest}]$

largest = r

if $i \neq \text{largest}$

swap[A[i], A[largest]]

Max-Heapify(A, largest)

حالت

max like

حالت

نیم

نیم

نیم

$$k' n + \log n + k$$

نیم

نیم

$$O(n^2) : \text{اگر } \frac{n}{2}$$

نیم

نیم

$$O(n \log n) : \text{اگر } \frac{n}{2}$$

نیم

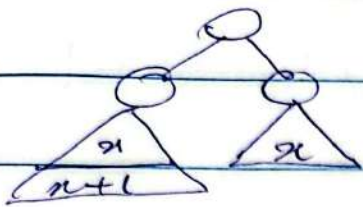
نیم

$$O(n \log n) : \text{اگر } \frac{n}{2}$$

نیم

نیم

آزمین



$$x = 2x_1 + 2$$

تعداد یابی

$$\rightarrow x = \frac{n-2}{2} \rightarrow x_{k+1} = \frac{x_{k-1}}{2} + 1 = \frac{x_{k-1}}{2} + \frac{x_{k+1}}{2}$$

$$\rightarrow T(n) = T\left(\frac{n}{2}\right) + O(1) \rightarrow T(n) \in \log_{\frac{n}{2}}(n)$$

$$T(n) = T\left(\frac{n}{2}\right) + O(1) + O(1)$$

$$= T\left(\left(\frac{n}{2}\right)^k\right) + k \cdot O(1)$$

تا وقتی که به 1

$$= T(1) + k \cdot O(1)$$

برسد جایگزین کنیم

$$\rightarrow \left(\frac{n}{2}\right)^k = 1 \rightarrow k = \log_{\frac{n}{2}}(n)$$

$$\rightarrow T(n) = T(1) + \log_{\frac{n}{2}} n \cdot O(1) = O(\log n)$$

ساخت heap به وسیله Max-heapify

Build-Max-Heap(A)

$i \rightarrow \text{heap size}$

for $i = \lfloor \frac{\text{heap size}(A)}{2} \rfloor$ down to 1

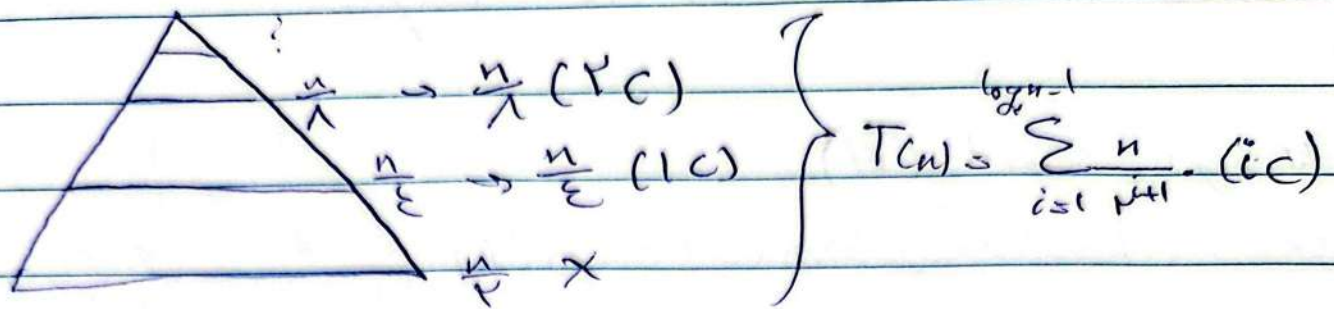
$i \rightarrow \text{heap size}$

Max-heapify(A, i)

حل می‌کند

$$\rightarrow O\left(\frac{n}{2} \times \log(n)\right) = O(n \log n)$$

با بررسی دقیق تر متوجه می‌شویم که این تابع $O(n)$ است.



$$\frac{n}{2^{i+1}} = 1 \rightarrow i+1 = \log_2 n \rightarrow \boxed{i = \log_2 n - 1} \quad i \text{ کی حد}$$

$$T(n) = C \cdot \frac{n}{2} \sum_{i=1}^{\log_2 n - 1} \left(\frac{1}{2}\right)^i \cdot i < C \cdot \frac{n}{2} \sum_{i=1}^{\infty} \left(\frac{1}{2}\right)^i \cdot i$$

$$< \frac{cn}{2} \cdot \frac{2}{1/2} = \frac{cn}{2} = O(n)$$

Build_max_heap(A, i) حالت بازگشت:

l = i, r = i + 1,

if l ≤ heapsize

Build_max_heap(A, l)

if r ≤ heapsize

Build_max_heap(A, r)

Max_heapify(A, i)

$$T(n) = 2T\left(\frac{n}{2}\right) + O(\log n)$$

آذین

Sort with Max Heap

Heap-sort (A)

Build-max-heap(A) $O(n)$

For $i = \text{heap-size down to } 2$ $O(n)$

swap(A[1], A[i]) $O(n)$

heapsize -- $O(n)$

Max-heapify(A, 1) $n \log n$

هر بار کوچکترین در آخر قرار میگیرد

$\rightarrow T(n) = O(n \log n)$

حذف و درج در heap (به عنوان) (تبرین) $(O(\log n))$ $\rightarrow$ $O(n \log n)$

Augmented Data Structures

در RBT را به گونه ای آید که مسئله بهر حل شود

Dynamic order statistics

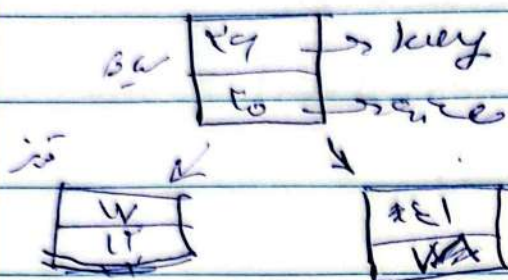
در D می خواهیم n امین کوچکترین کدام است؟
(۲) رتبه (rank) یک عنصر

(اطلاعات می تواند کم و زیاد شود)

در $W(k)$ درج خطی است و در $W(k)$ گنارینه به از
RBT استفاده کنیم

آذین

مجموعه n به مقدار کم یا زیاد نسبت به n (نمره Null میزنند)
(در n هم نمره میزنند)



$$n_{size} = n_{l.size} + n_{r.size} + 1$$

$$T.Null.size = 0$$

پس n_{size} هر گاه بازگشت حساب می شود

برای پیدا کردن جواب از n_{size} استفاده می کنیم. اگر رتبه ۱۳ را بخواهیم به $T.l.size$ نگاه می کنیم، چون $13 \leq 14$ ، پس انتخاب می شود. اگر $13 > 14$ بود به $T.r.size$ بازگشت می کردیم زیرا $13 \leq 15$ است. پس در نهایت

OS_Select(x, i):

$$r = n.l.size + 1$$

$$\text{if } i \leq r$$

return n

else if $i \leq r$

return OS_Select($n.l, i$)

else

return OS_Select($n.r, i - r$)

فقط رتبه که کمتر یا مساوی r باشد میزنند

به طوری که $O(\log n)$ است. (ارتقاء رتبه)

آزمین

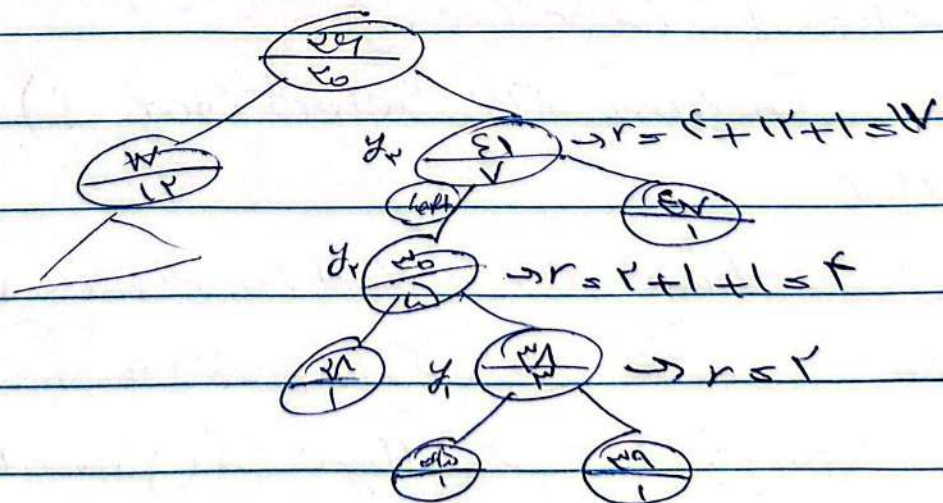
$$r = x.\text{left.size} + 1$$
$$y = x$$

white $y \neq T_{root}$

if $y = z \cdot p \cdot w$ let

$$r = r_{\text{left.size}} + 1$$
$$y = y \cdot p$$

return r; $\rightarrow O(\log n)$

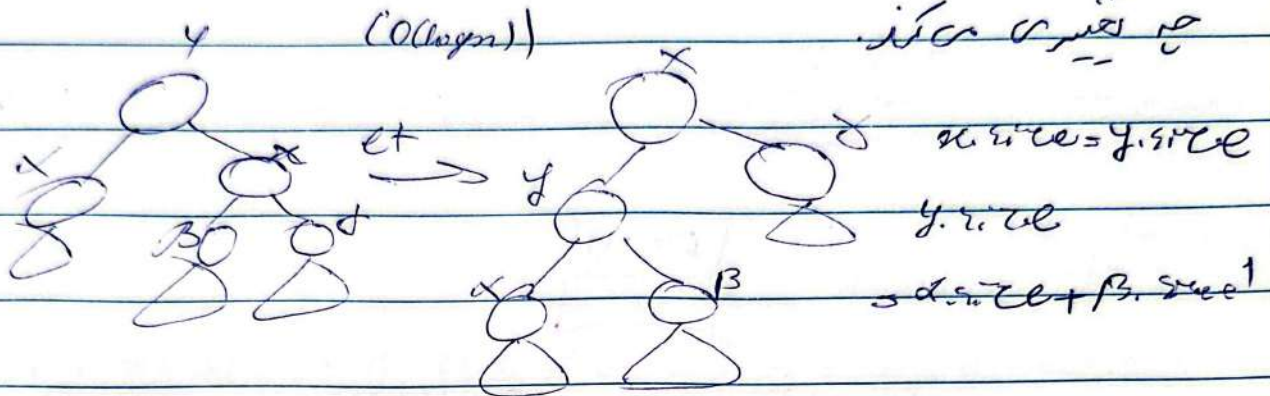


آفرین مهر

به سبب این size در RBT

برای insert هر گاه که در مسیر پیدا کردن محل درج با $size$ مواجه شد
بسیار برای $fixup$ یا به معنی کردن $left rotate$ و $right rotate$ سایر

به تغییر می کند.



برای delete هم نام که داریم باید که $size$ را در نظر بگیریم و $fixup$ که rotation ها به همین علت است.

نکته کاربردی RBT

Interval Tree (باز / بسته / نیم باز)

Overlap $\rightarrow j.high \geq i.low$ and $j.low \leq i.high$

$j.high < i.low$ (i سبج)

$j.low > i.high$ (i داخل j)

علاوه بر این Interval Tree

همچنین در Tree جستجو

آذین

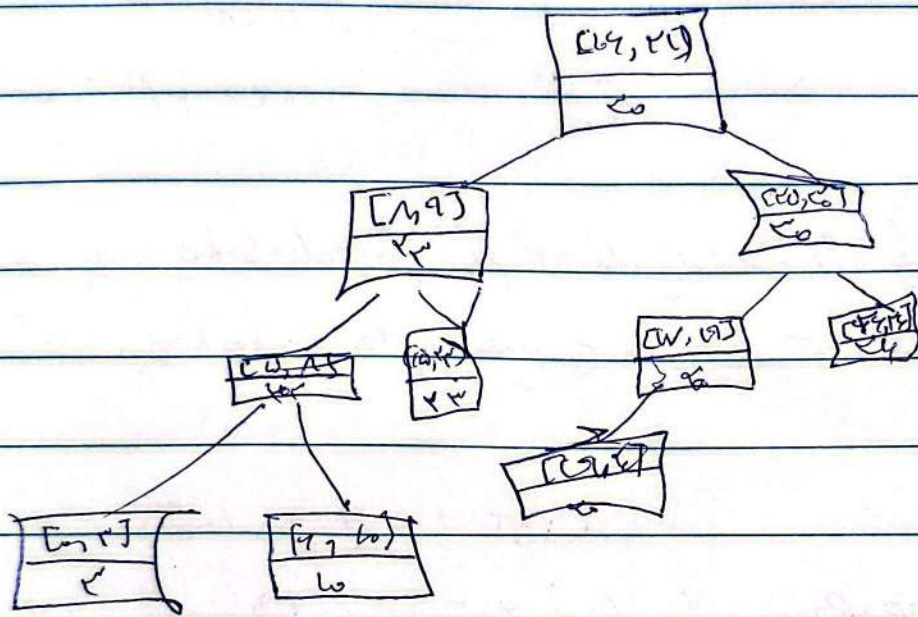
خوبه به جویک باز در T است که با i اشتراک دارد.

چون بر اساس low باز است. $key = x.right.low$

$x \rightarrow max \rightarrow max(x.left.max, x.right.max, x.val.max)$



سال:



در اینجا، هیچ نقطه max از آرایه نبود.

Interval Tree

interval_search(T, i)

$x = T.root$

while $x \neq T.null$ and i not overlap x with

| if $x.left \neq T.null$ and $x.left.max \geq i.low$

$x = x.left$ (چون به چپ می‌رویم)

else $x = x.right$

return x ;

آزیم

اثبات دست کار کرد

کثرت: اگر T شامل بازه‌ای باشد که low و high در آن باشد
زیربافت T به low و high بازه است. (در حال نگه داشتن)
اثبات بر این است که:

باید استوار است. $\rightarrow T \text{ meet}$ low و high استوار
کدام استوار نیست

$\text{low} \leq \text{high}$
(اگر $\text{low} > \text{high}$)

تنگنا $\text{low} \leq \text{high}$ و $\text{low} > \text{high}$ است که low و high برقرار است. (چون low و high در آن باشد)
(بوده باید باشد)

اگر $\text{low} \leq \text{high}$

باید low و high شامل بازه‌ای باشد که low و high در آن باشد
بهال حقیقت: فرض کن low و high در آن باشد.

$\text{low} \leq \text{high}$ (فرض می‌کنیم)

یعنی low و high بازه‌ای است که low و high در آن باشد.

low و high

اگر low و high در آن باشد

$\text{low} < \text{high}$

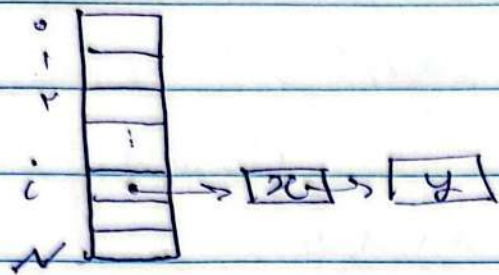
$\text{low} < \text{high}$ و $\text{low} > \text{high}$

به دلیل low و high در آن باشد (تناقض)

تکلیف زائد: $O(\log)$

آزمین

Hash Table



$i = \text{Hash}(X)$
 $i = \text{Hash}(Y)$ } conflict
 $\text{arr}[i] \leftarrow \text{add}(x, y)$

Insert Hash Table (T, x)

$i = \text{Hash}(X)$

Add To List (T[i], x)

Search Hash Table (T, x)

$i = \text{Hash}(x)$

If T[i] == Null

return Null;

else

return Find_In_List (T[i], x)

Time Complexity: $O(1)$ for search, $O(n)$ for insert

طراحی الگوریتم

۱) روش Divide & Conquer یا تقسیم و فتح

۲) برنامه ریزی پویا Dynamic programming

۳) روش حریص Greedy

بازگشت Divide & conquer

۱. کار تقسیم: تقسیم مسئله اصلی به چند زیر مسئله
۲. کار حل بازگشت: حل کردن بازگشت زیر مسائل
۳. کار ترکیب: ترکیب جواب زیر مسائل کوچک و به دست آوردن جواب اصلی

مسئله: Maximum (A, n): غیر بازگشتی

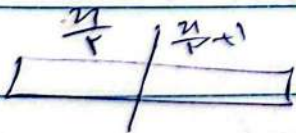
$max = A[1]$

for i is 2 to n do

| if $A[i] > max$

$max = A[i]$

return max



$max1 = max(0, \frac{n}{2} - 1, A)$

$max2 = max(\frac{n}{2}, \overbrace{size-1}^{n-1}, A)$

return $max1 > max2 ? max1 : max2$

لر بازگشتی

۱) تقسیم به دو قسمت با $\frac{n}{2}$ عضو

۲) حل با بازگشتی بازگشت

۳) ترکیب

آزیر

Maximum-DC (A, i, j):

if $i \leq j$ } $\rightarrow$ بازگرداندن برای $i=j$
 return A[j]

$k = \lfloor \frac{i+j}{2} \rfloor$ $\rightarrow$ تقسیم

max 1 = Maximum-DC (A, i, k) } $\rightarrow$ باز

max 2 = Maximum-DC (A, k+1, j) } $\rightarrow$ باز

return Max {max 1, max 2} $\rightarrow$ باز

$$T(n) = r T\left(\frac{n}{r}\right) + O(1) = O(n)$$

$$C(n) = r [r T\left(\frac{n}{r}\right) + C] + C$$

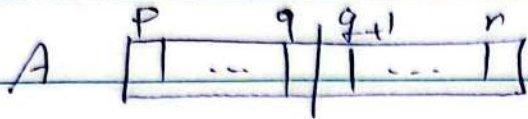
$$= r [r (r T\left(\frac{n}{r}\right) + C_1) + C_r] + C_n$$

$$= r^k T\left(\frac{n}{r^k}\right) + \underbrace{(r^{k-1} + r^{k-2} + \dots + r^0) C}_{\sum_{i=0}^{k-1} r^i C}$$

$$r^k = n \quad \rightarrow \quad k = \log_r n \quad \rightarrow \quad T(n) = n T(1) + \sum_{i=0}^{\log_r n} r^i$$

$$= n \cdot C + C \cdot \frac{r^{\log_r n + 1} - 1}{r - 1} = n C + C(n-1) = O(n)$$

مرتب کرنا: 3 روش از کسب



د کسب

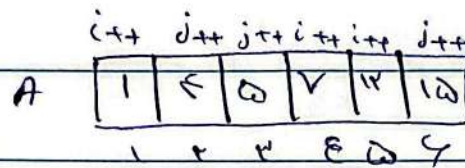
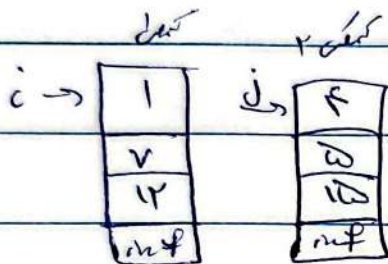
$$q = \left\lfloor \frac{p+r}{2} \right\rfloor$$

Merge-Sort(A, p, q)

حل از کسب

Merge-Sort(A, q+1, r)

ترکیب



A را بازسازی کنیم
(الگوریتم Merge)

Merge-Sort(A, p, r)

if p == r

return;

Merge-Sort(A, p, q)

Merge-Sort(A, q+1, r)

merge(A, p, q, r) → O(n)

$$T(n) = T\left(\frac{n}{2}\right) + O(n) = O(n \log n)$$

آذین

merge sort ال

Merge (A, p, q, r)

$$n1 = p - q + 1, \quad n2 = r - q$$

for i = p to q do

$$L[i - p + 1] = A[i]$$

$$L[n1 + 1] = \infty$$

} صرف آرایه اول $O(n1)$

for i = q + 1 to r do

$$R[j - q] = A[j]$$

$$R[n2 + 1] = \infty$$

} صرف آرایه دوم $O(n2)$

$$i = j = 1$$

} $O(1)$

for k = p to r do

$$\text{if } L[i] < R[j]$$

$$A[k] = L[i]; i++;$$

else

$$A[k] = R[j]; j++;$$

} $O(n)$

$$\therefore T(n) = O(n)$$

merge sort (تکرار) merge sort

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1) + O(n)$$

$$= O(n \log n)$$

quick-sort الگوریتم دیگر است که مبتنی بر تقسیم و تسطیح است.

آذین

quick-sort (pivot (randomly selected))

array: $a_1, \dots, a_n$ $O(n)$

↓ $Q(A_l); Q(A_r);$ $\rightarrow$ تمام ترتیب زار

Quick-sort (A, p, r)

if $p = r$ return;

$q = \text{partition}(A, p, r)$ // انتخاب پویا و تقسیم $O(n)$
// pivot را به جا آورند

Quick-sort ($A, p, q-1$)

Quick-sort ($A, q+1, r$)

$$T(n) = T(k) + T(n-k-1) + O(n)$$

اگر p به صورت تصادفی انتخاب شود (یعنی q میان اعداد باشد) $T(n) = O(n \log n)$
و در بدترین حالت که $q = k$ باشد:

$$T(n) = T(n-1) + O(n) = O(n^2)$$

که میان مقادیر نزدیک به $n/2$ باشد.

به این علت است که Quick-sort به کاربرد است که به مقادیر خاص $n \log n$ می‌رسد.
انتخاب پویا pivot. مقادیر q که به ترتیب ضریب ثابت تابع کمتر از n شود کمتر
از دیگر است.

Dynamic Programming به روش DP

در این روش حل هر بازگشت حافظه دارد تا مسئله تکراری دوباره حل نشود.
به این روش Memoized DP میگویند. (DP حافظه دار)

اما DP یک بهبود دیگر انجام می‌دهد.

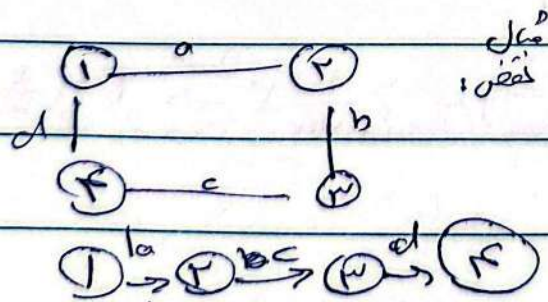
DP زیرمسائل تکراری را ذخیره می‌کند. (Memoized)
(۲) به وسیله optimal substructure تعداد زیرمسائل را کاهش می‌دهد. (لتریداً به حداقل نمی‌رسد.)

یک مسئله ممکن است دارای optimal substructure باشد یا نباشد.

مسئله بهینه‌سازی است که سوالی که دنبال بهترین جواب هستیم مانند پیدا کردن کوتاه‌ترین مسیر از رأس s به t در گراف. (با این کار مثل در)

بین قطع حالت

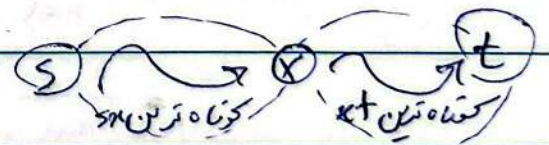
(۲) مسئله بلندترین مسیر ساده



کوتاه‌ترین ۲ تا ۴ نیست.

لینک نمی‌تواند دو زیرمسئله را ترکیب
بهینه کردن چون لزوماً به جواب اعلان نمی‌رسد.

(۱) مسئله کوتاه‌ترین مسیر



ادغام شده و زیرمسیر دارد شده s و t

کوتاه‌ترین است. (با به هم خلیف ثابت می‌شود)

در مجموع $3+3$ مسیر با به هم خلیف می‌شود

ولی ما $3+3$ مسیر را به هم خلیف می‌کنیم.

(لینک دیگر نمی‌تواند را دارد ترکیب بهینه)

آذین مهر

حل مسئله پراکنش گذارش منب ماتریس ها با روش D.P.

$$M_1 \times M_2 \times \dots \times M_n$$

شرکت پخش وجود دارد اما پراکنش گذارش ها مختلف باعث می شود تعداد منب ها مختلف انجام شود. (به علت ابعاد ماتریس ها)

$$A_{p \times q} \times B_{q \times r} = C_{p \times r}$$

→ عا به هر خانه q منب نیاز دارد. $q-1$ جمع نیاز دارد.

→ کل منب ها $p \times q \times r$

→ کل جمع ها: $p \times r \times (q-1)$

تعداد کل پراکنش گذارش معتبر آخرین منب

$$(M_1 \times M_2 \times \dots \times M_k) (M_{k+1} \times M_{k+2} \times \dots \times M_n)$$

$$P(n) = \sum_{k=1}^{n-1} P(k) \cdot P(n-k) = C_{n-1} \quad (\text{کاتالان } (n-1))$$

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

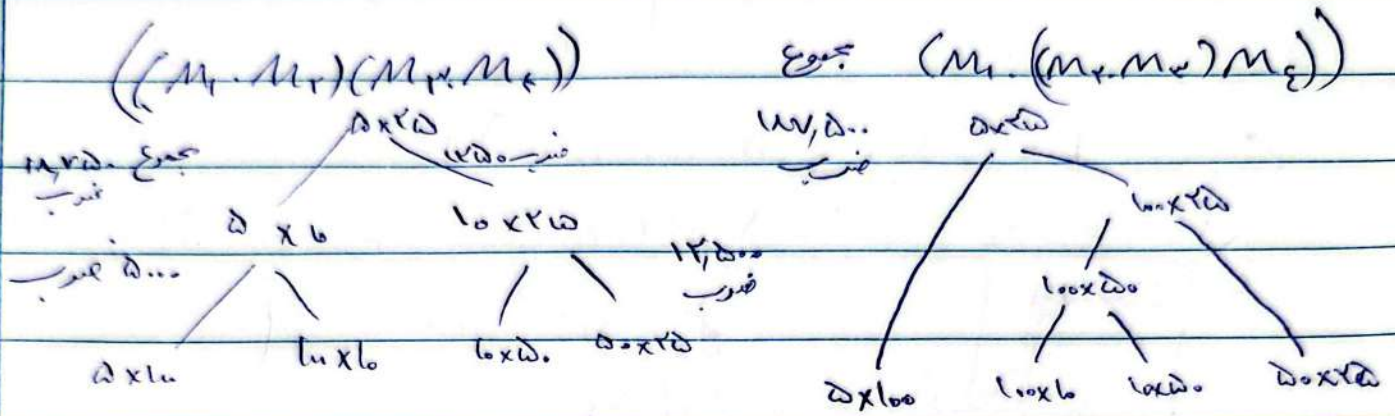
$$= \frac{1}{n+1} \left(\frac{(2n)!}{(n!)^2} \right)$$

$$= \frac{1}{n+1} \underbrace{\left(\frac{n+1}{1} \right) \left(\frac{n+2}{2} \right) \dots}_{\sum 2} \geq \frac{1}{n+1} \cdot 2^n \rightarrow O\left(\frac{2^n}{n}\right)$$

روش DP برای مسئله پرانتزگذاری ماتریس ها

$$M_1 \times M_2 \times M_3 \times M_4$$

$$5 \times 10 \quad 10 \times 6 \quad 6 \times 5 \quad 5 \times 20$$



حالت کمترین وقت است که بعد بزرگتر از ضرب باشد حذف شود اما می توانه اینکه به نایاب (مثل نقض) به روش Greedy

$$(M_i \dots M_k) (M_{k+1} \dots M_j) \rightarrow C_{ij}$$

$$Q_{i,k} \times Q_{k+1,j}$$

ضرب آخر

معادله بازگشتی

$$C_{i,j} = C_{i,k} + C_{k+1,j} + d_{i-1} \cdot d_k \cdot d_j$$

ضرب آخر ن - ز حالت دارد.

در بلور هم حل کنیم و min می گیریم. (از عدد کانال) بهترین حالت

ها کمتر می شود، چون داریم از و اما استفاده می کنیم و حالت

به این از ضرب آخر یک است (ترکیب بهینه دارد).

$$C_{ij} = \min_{k \in [i, j-1]} \{ C_{i,k} + C_{k+1,j} + d_{i-1} \cdot d_k \cdot d_j \}$$

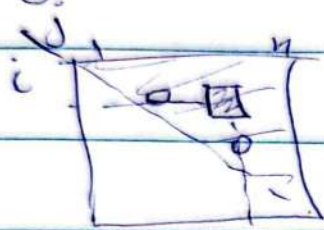
ز چاک

آزمین

حافظه که داریم جواب

در ممکن است در حالت های مختلف زیر مسئله نگاریم و درستی
باشیم پس ذخیره درستی هم است.

$$2 + \binom{n}{2} \leq n \leq n \leq 1 \leq n \leq n$$



پس حداکثر یک ماتریس $n \times n$ حافظه کافی است.

نصف ماتریس کافی است چون $n \times n$ $n \times n$

باید طوری شروع کنیم که زیرمسائل کمترین کار را بقیل حل شود

در حل هر یک از ماتریس $n \times n$ گوییم راست بریم

جواب اصلی است.

پس ابتدا حالت های که (n, n) در هر یک از این موارد و به ترتیب به

حالت های $n \times n$ به تدریج به سمت $n \times n$ می رود

در DP معمولاً از مسئله بزرگ پایین به بالا و به ترتیب ترکیب

در DP از کوچکترین به بزرگترین و به ترتیب ترکیب

در DP از کوچکترین به بزرگترین و به ترتیب ترکیب

$$C_i, i = 0$$

آزین

$M_1 \quad M_2 \quad M_3 \quad M_4$

$10 \times 10 \quad 10 \times 10 \quad 10 \times 10 \quad 10 \times 10$

John

$i \backslash j$	1	2	3	4
1	0	$\Delta_{1,2}$	$\Delta_{1,3}$	$\Delta_{1,4}$
2		0	$\Delta_{2,3}$	$\Delta_{2,4}$
3			0	$\Delta_{3,4}$
4				0

$$C_{1,2} = C_{11} + C_{22} + d_{1,2}d_{2,1}$$

$$= 0 + 0 + \omega \times 10 \times 10 = \omega$$

$$C_{2,3} = 0 + 0 + \omega = \omega$$

$$C_{3,4} = 10 \times \omega = \omega$$

$$C_{1,3} = \min \begin{cases} \text{kas 1} & C_{11} + C_{33} + d_{1,3}d_{3,1} = \omega_{0,000} \\ \text{kas 2} & C_{12} + C_{23} + d_{1,2}d_{2,3} = \omega_{0,000} \\ \text{kas 3} & C_{14} + C_{43} + d_{1,4}d_{4,3} = \omega_{0,000} \end{cases}$$

$$\omega_{0,000} = \omega \times 10 \times 10 = \omega$$

$$C_{2,4} = \min \begin{cases} \omega_{0,000} \\ \omega_{0,000} \end{cases}$$

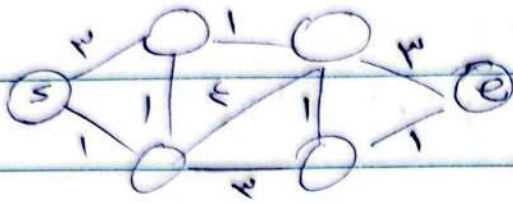
$$C_{1,4} = \begin{cases} \text{kas 1} & C_{11} + C_{44} + d_{1,4}d_{4,1} = \omega_{0,000} \\ \text{kas 2} & C_{12} + C_{24} + d_{1,2}d_{2,4} = \omega_{0,000} \\ \text{kas 3} & C_{13} + C_{34} + d_{1,3}d_{3,4} = \omega_{0,000} \end{cases}$$

زبان الترميز $\Rightarrow 1 + 2 + n = \frac{n(n+1)}{2}$

$$\Rightarrow T(n) = \frac{n(n+1)}{2} O(j-i) = O(n^3)$$

آذین

حل مسئله گرلف با DP - پیدا کردن کوتاه ترین مسیر



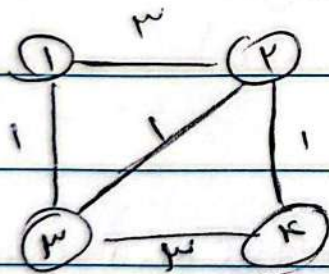
مسیر 5 به 6 طوری که مجموع وزن
بال ها حداقل باشد.

DP

Floyd-Warshall → از هر رأس به هر رأس → all-pairs
Dijkstra → از یک رأس به بقیه → single-source
Greedy

single-source → فرض بر این است که وزن منفی نداریم.

Floyd-Warshall → دور منفی (مجموع وزن ها کمتر از صفر) نداریم.

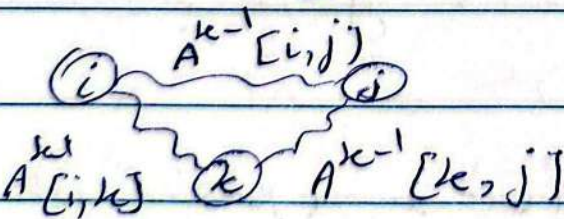


فقط از 1 به 4

A^0 این است

	1	2	3	4
1	0	3	1	∞
2	3	0	1	1
3	1	1	0	3
4	∞	1	3	0

مثال



$$A^k[i, j] = \min \begin{cases} A^{k-1}[i, j] \\ A^{k-1}[i, k] + A^{k-1}[k, j] \end{cases}$$

پایه مثال) از روش آنگاه $A^n[i, j] : A[i, j]$ به توالی ساخته کرد

$$k \text{ است } A^n[i, j] = \min \{ A^n[i, k] + A^n[k, j] \}$$

کدام

0	2	1	4
2	0	1	1
4	1	0	3
8	1	3	0

در مرحله اول به این شکل به هم
 گره ها میزنند چون گره ها به هم
 در هر خانه خطی را میگیریم و اگر کمتر از قبلی
 بعد حالت جدید را می نویسیم.

کدام

0	2	1	4
2	0	1	1
1	1	0	3
4	1	2	0

کدام

0	2	1	3
2	0	1	1
1	1	0	2
3	1	2	0

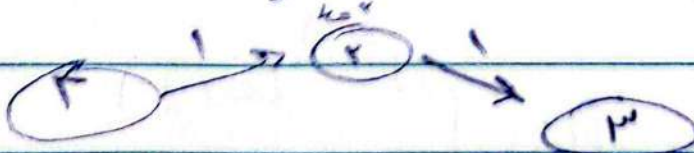
$k=n=4$

0	2	1	3
2	0	1	1
1	1	0	2
3	1	2	0

طول کوتاه ترین مسیر از
 هر یک به هر یک

خود کوتاه ترین مسیر طبق 2 کدام به دست می آید

مثال 4 به 3 است



$$T(n) = O(n^2(n+1)) = O(n^3)$$

آذین مهر

روش حریصانه Greedy (بهترین DP)

برخلاف DP همه حالت‌ها را حل نمی‌کنیم، با یک استدلال مشخص
می‌دهیم کدام بهتر است و آن را حل می‌کنیم. (Greedy choice)
لکه بنابر این در هر زیرمسئله فقط یک انتخاب انجام می‌شود

پس از ابتدا دنبال greedy-choice می‌گردیم.

مسئله چند کردن اسکناس با سکه‌ها (کمترین تعداد سکه)
اسکناس ۲۸ تومنی

سکه ۵، ۱۰، ۲۰، ۵۰ (به تعداد کافی)

(حریصانه) ۵ سکه $\rightarrow$ ۱، ۲، ۵، ۱۰، ۵۰ $\rightarrow$ ۲۸

سکه ۵، ۱، ۶، ۷، اسکناس ۱۰۰

(۱) $3 \times 7 + 7 = 18$ حریصانه

(۲) $3 \times 6 = 18$ بهینه

پس انتخاب حریصانه به اثبات نیاز دارد.

مسئله کوتاه ترین مسیر Dijkstra (به سبب Greedy)

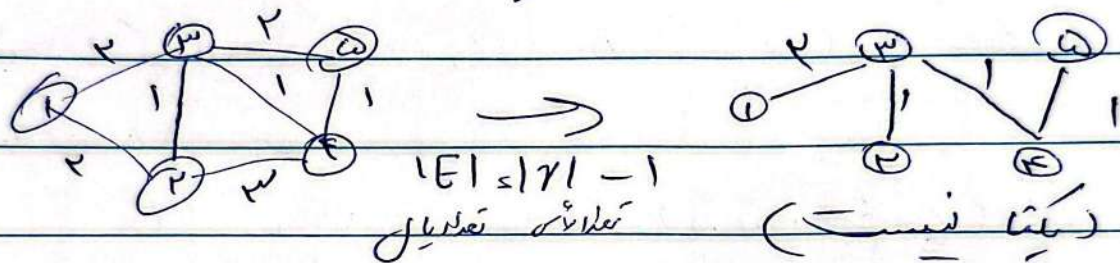
مسئله یافتن کوتاه ترین گره (Min spanning tree)

ما می خواهیم یک زیرگراف از گراف وزن دار G پیدا کنیم که:

۱. رخت باشد (گراف همبند بدون حلقه)

۲. یونیک باشد (رأس هر رأس)

۳. مجموع وزن یال ها کمینه شود



الگوریتم حریصانه با انتخاب یال با کمترین وزن

فصل P_{min}
در هر گره به صورت گشایی نشود $\leftarrow$ $lenuskeel$ الگوریتم بهینه

روش $lenuskeel$: انتخاب یال با کمترین وزن

شرطی که بعد نشود تا جایی که رأس ها تمام شود

که آن را به صورت P الگوریتم بهینه است

آذین مهر

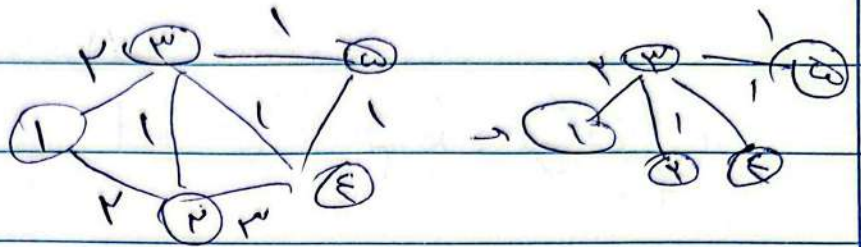
$$V(3, 4) = 1 \quad \downarrow \quad (عاشق - 1 - 1) \text{ عاشق}$$

$$V(3, 4) = 1$$

$$V(4, 5) = 1$$

$$\cancel{V(3, 2) = 1}$$

$$V(1, 3) = 2$$



$$\frac{V(1, 2) = 2}{V(3, 4) = 3}$$

اینست مورد این الگوریتم بهینه است

$$\text{sort} \rightarrow |E| \cdot \log |E|$$

یک بار DFS میزنیم و اگر به رأس جدید برسیم $\rightarrow$ Loop exist
در تسلسل سگدا.

یعنی قبل از اضافه کردن (u, v) یک $DFS(F, u)$ میزنیم و اگر به v برسیم در دارد.

$$\begin{aligned} O(|V|^2) & \text{ عاشق عاشق} \\ O(|E| + |V|) & \text{ لیست عاشق} \end{aligned} \quad \begin{aligned} & \rightarrow \begin{cases} O(|V|^2) \\ O(2|V|) = O(|V|) \end{cases} \quad (|E| < |V| - 1) \\ & \downarrow \text{ بهتر است.} \end{aligned}$$

$$T(u) = T(\text{sort}) + T(\text{find-loop}) \cdot O(|E|)$$

$$= O(\underbrace{E \log E}_{O(r^2 \log r)}) + E \cdot \underbrace{O(r)}_{O(r^3)} = O(r^3) = O(E \cdot r)$$

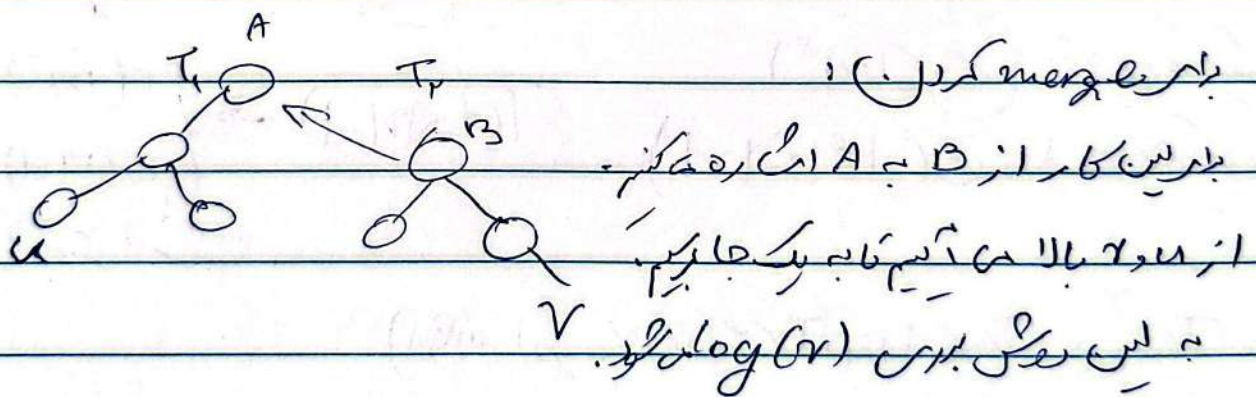
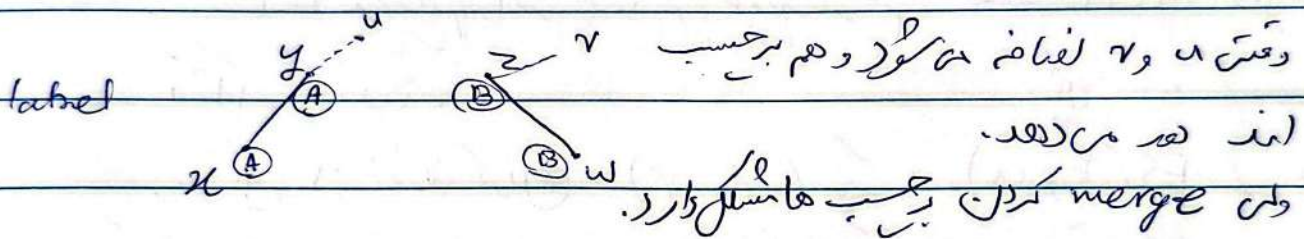
آزاد

حفظ در صورتی حلقه تشکیل می شود که دو رأس متفاوت باشند
از قبل باشند و هر دو از قبل برای یک درخت باشند.
به این روش زمان اجرا بهتر می شود.
در درخت در حلقه را یک set می گیریم و یک set می گیریم.

اگر u و v عضو یک set باشند حلقه تشکیل می شود و اگر نه غیر.

set DS

→ Add, del, find, union, intersect



$$T_{un} = O(E) \cdot \log E + O(E) \cdot \log(V) = O(E \log(V))$$

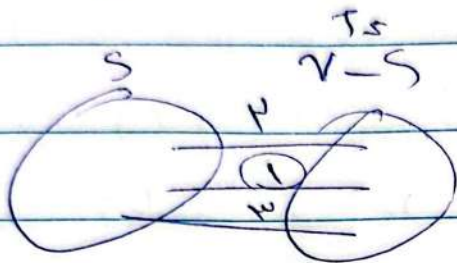
الگوریتم هاشنگ

آزمایش

الگوریتم $prms$ در مجموع

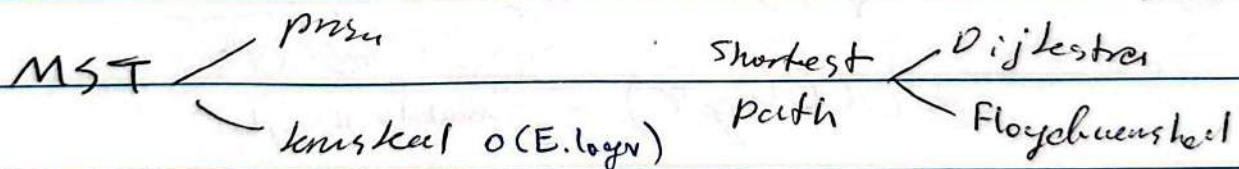
از بین V بهای که در T قرار دارد و S و $V-S$ از V وجود دارند، V که کمترین وزن دارد را انتخاب می‌کنیم.

(greedy choice)



پس سؤال اصلی از ده نقاط
جواب می‌کنیم و min را می‌گیریم.

۱. رأس دلخواه u را در S قرار ده و $T = V-S$
 ۲. از بین V بهای که T و S کمینه را بگیرد (فقط V و u)
 ۳. حال V را از T جدا کرده و S اضافه کن و مرحله را تکرار کن.
- (تا جایی که T تهی شود.)



زمان $prms$: $1 - |V|$ مرحله دارد، در هر مرحله V که کمینه انتخاب می‌شود یک $linear search$ با $O(E)$ است. $\rightarrow O(|V| \cdot |E|)$

بهتر است از $MinHeap$ استفاده کنیم:

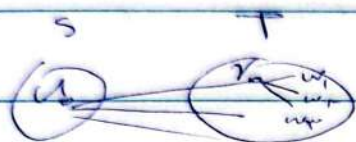
آزمایه

حالت اولیه $S = \{u\}$

$$O(\text{degree}(u)) = O(|V|) \quad \text{زمان یافتن حالت اولیه}$$

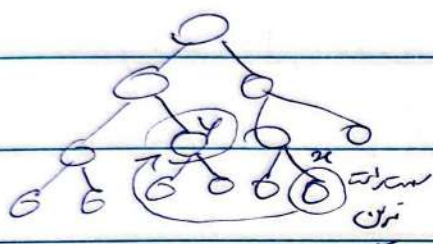
در هر گام یال (u, v) از یال های بین S و T حذف

و یال های متصل به v به S (یا T) اضافه می شود.



بین (S, T) اضافه می شود.

بین (S, T) حذف x ، $\text{degree}(v) - 1$ اضافه



به سمت بالا یا پایین

حذف $O(k \cdot \log E)$ heapify

(به سمت بالا یا پایین)

اضافه $O((\text{degree}(v) - k) \cdot \log E)$ (به سمت پایین یا بالا می گذاریم و بلافاصله به سمت بالا می آوریم)

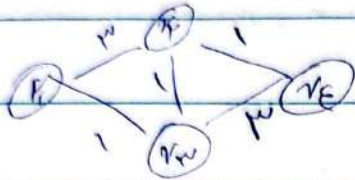
زمان update در مجموع: $O(\text{degree}(v) \cdot \log E)$

$$\begin{aligned} \text{زمان کل} &= \sum_v \text{degree}(v) \cdot \log E + \underbrace{O(|V|)}_{\text{زمان اولیه}} \\ &\leq O(E \log V) \quad \text{زمان کل} \end{aligned}$$

مزیت پویش لین است که نیاز نیست همه یال ها را (با $\min$ کمترین)

توجه: $d(\log E) = O(\log V^2) = O(2 \log V) = O(\log V)$

الگوریتم Dijkstra



باینرفن دانش من منشی

1	2	3	4
D	-	3	1

با وزن یال مستقیم از شروع به یال آن مقدار می دهیم.

رأس w طویل انتخاب شود که $D(w)$ کمترین باشد.
 در $D(w)$ منشی $D(v)$ و $D(w) + c(w, v)$ را مقایسه می کنیم.

$$D^i(v) = \min \{ D^{i-1}(v), D^{i-1}(w) + c(w, v) \}$$

یعنی در هر مرحله تمام یال های w مانند v را مقایسه می کنیم.
 و یک می کنیم تا به کمترین مسیر کوتاه تر رسید یا خیر.

1	2	3	4
D	-	2	1

هر کدام یکبار w بود و کنار می گذاریم.

1	2	3	4
D	-	2	1

خود کوتاه ترین مسیر کوتاه $w = 2$ در هر مرحله تعیین می شود.

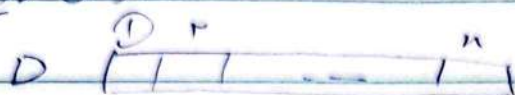
(جواب نهایی فقط برای رأس مانند v)
 که w ندارد.

آذین

رأس شروع

Dijkstra

تحليل زمني



∞ یا فنک یال v, w $D[v] = A[s, v]$ مقدار اولیه

w رأس است که کمترین $D(w)$ را دارد.

$D[v] = \min_{قبل} \{ D[v], D[w] + A[w, v] \}$ وزن $w-v$ جدید

پس $n-2$ مرحله $(n-2)$ (به جز اول و آخر) رأس انتخاب می شود.

در هر مرحله در $O(1)$ می توان $D(v)$ جدید را گرفت.

$\text{degree}(w) \times O(1) = O(\text{degree}(w))$ $\rightarrow$ اهمیت w

به علاوه در هر مرحله

برای پیدا کردن w در آرایه D کمینه می گیریم.

① جست و جوی خطی $O(n)$

زمان کل: $(n-2) O(n) + \sum_w O(\text{degree}(w))$

$= O(n^2) + O(E) = O(n^2)$

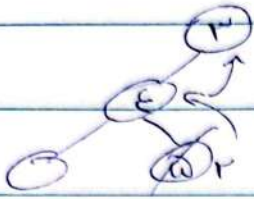
② جست و جوی با MinHeap

ساخت heap اولیه $O(n)$ $O(\log n)$

زمان پیدا کردن کمینه $O(1) + \text{heapify}$

آذین

دقت مقدار O عمل شود در heap هم باید تغییر کند



heapify از گره به ریشه لازم است. $O(\log r)$

برای همه حساب ها $O(\text{degree } W \cdot \log r)$

← زمان کل

$$O(n) + \sum_{v \in V} O(1) \cdot O(\log r) + \sum_w \text{degree } w \log r$$

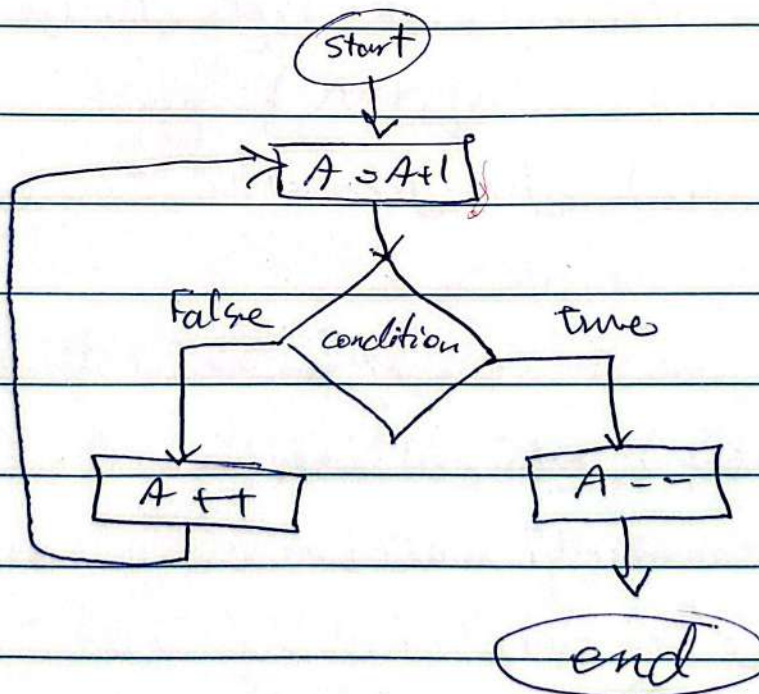
جدا غالب

$$= O(r) + O(r \log r) + O(E \log r)$$

$$= O(E \log r) \quad \text{زمان Dijkstra}$$

$\downarrow$
 $\downarrow$
 r r
(غالب هم بند)

فلجیاریت



آذین